

Unit-3: Digital Electronics

CONTENT

1. Introduction to Digital Electronics

- 1.1.Need of Digital Electronics
- 1.2.Analog v/s Digital Signal

2. Number System and Representation

- 2.1.Introduction
- 2.2.Binary to Decimal Conversion
- 2.3.Decimal to Binary Conversion
- 2.4.Binary to Hexadecimal Conversion
- 2.5.Decimal to Hexadecimal Conversion
- 2.6.Hexadecimal to Decimal Conversion
- 2.7.Octal to Decimal Conversion

3. Basic and Universal Logic Gates

- 3.1.Introduction
- 3.2.Logic Addition
- 3.3.Logic Multiplication
- 3.4.Logic Gates
 - 3.4.1. OR Gate
 - 3.4.2. AND Gate
 - 3.4.3. NOT Gate
 - 3.4.4. NAND Gate
 - 3.4.5. NOR Gate
 - 3.4.6. EXOR Gate
 - 3.4.7. EXNOR Gate
- 3.5.Logic Realization of Logic Gate Using Universal gates
 - 3.5.1. Implementation of NOT, AND, OR, NOR, NAND, EX-OR, & EX-NOR Gate Using Universal gates

4. Boolean Algebra Simplification of Boolean Function

- 4.1. Boolean Algebra
- 4.2. Laws of Boolean Algebra
- 4.3. Duality in Boolean Algebra
- 4.4. De Morgan's Theorem
- 4.5. Boolean Algebra Theorems
- 4.6. Simplification of Boolean expressions
- 4.7. SOP and POS Representation for Logic Expressions

5. Karnaugh map (K-Map)

- 5.1. Introduction
- 5.2. Rules of Minimization in K-Map
- 5.3. Two-Variable K-Map
- 5.4. Three-Variable K-Map
- 5.5. Four-Variable K-Map

6. Binary Arithmetic

7. Complements

- 7.1. Subtraction using complements (1s & 2s)

8. Codes (Weighted and Non-weighted codes, BCD & Gray Code)

9. Sampling & Quantization in Digital Signal Processing

10. Fundamentals and Applications of Quantum Computing and Edge Computing

11. Summary

12. Previously asked Questions in AKTU Exam

- 12.1. Short Answer Questions
- 12.2. Long Answer Questions
- 12.3. Numerical Problems

13. Model Questions (Theory & Numerical)

Prepared by: *Mr. Hitesh Tomar*

Reviewed by: *Mr. Mudit Saxena*

Note: *Handouts provided are supplementary materials. Students are advised to refer the prescribed text and reference books for comprehensive understanding.*

Digital Electronics is the sub-branch of electronics that deals with digital signals for processing and controlling various systems and sub-systems. Digital electronics are the most common representation of Boolean algebra and are the basis of all digital circuits for computers, mobile phones, and numerous other consumer products. In various applications like sensors and actuators, usage of digital electronics is increasing extensively.

The most common fundamental unit of digital electronics is the logic gate. By combining numerous logic gates (from tens to hundreds of thousands) more complex systems can be created. The complex system of digital electronics is collectively referred to as a digital circuit.

1.1 Need of Digital Electronics

The more accurate representation of the digital signal can be generated using more binary digits and has a unique way of representation. With the support of computer handled digital systems, trending functions can be fed to software revision and required no changes in hardware. Moreover, the immunity to noise in the digital system allows the data to be retrieved and stored without any degradation. In an analog system, the aging noise may vary due to wear and tear properties in the stored information. But in digital, if the noise is below the suggested level, the data can be restored perfectly. Even if there is any presence of significant noise, the redundancy use allows the restoring of actual data and provides a strong resistance to errors.

1.2 Analog vs Digital Signal

Almost all the signals in the World are analog i.e. they are continuously varying values. There are lots of continuously variable signals or simply analog signals in nature like light, motion, sound, temperature, pressure etc.

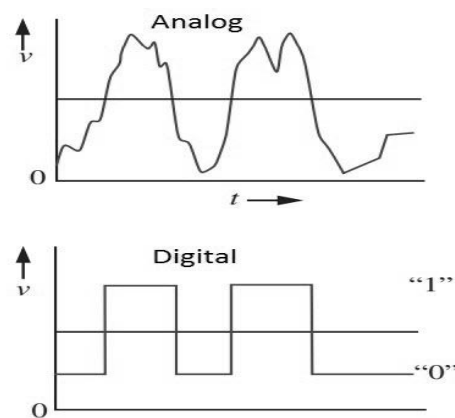


Figure 1.1 Representation of analog and digital signal

Digital Signals vary in discrete levels, in contrast to the continuous representation of analog signals. Generally, the discrete levels in a digital signal are just two values: ON and OFF. Even though all the physical signals of the nature are continuous analog signals, representing signals as discrete values has its own advantages.

Traditionally, all electronic devices processed analog information only. But, the development in technology has led to using digital signals for easy processing and such techniques are called Digital Signal Processing.

2. Number System and Representation

2.1 Introduction

In digital electronics, the number system is used for representing the information. The number system has different bases and the most common of them are the decimal, binary, octal, and hexadecimal. The base or radix of the number system is the total number of the digit used in the number system.

Table 2.1. Characteristics of Commonly used Number Systems

Number System	Base or radix(b)	Symbols used (d _i or d _{-j})	Weight assigned to position		Example
			i	-j	
Binary	2	0,1	2 ⁱ	2 ^{-j}	1101.11
Octal	8	0,1,2,3,4,5,6,7	8 ⁱ	8 ^{-j}	3647.53
Decimal	10	0,1,2,3,4,5,6,7,8,9	10 ⁱ	10 ^{-j}	3874.65
Hexadecimal	16	0,1,2,3,4,5,6,7,8,9, A, B, C, D, E, F	16 ⁱ	16 ^{-j}	4DAC.54

2.2 Binary to Decimal Conversion

Conversion of binary to decimal (base-2 to base-10) numbers and back is an important concept to understand as the binary numbering system forms the basis for all computer and digital systems.

Examples:

- Convert binary (10101)₂ to decimal.

Solution:

$$10101_B = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 4 + 1 = 21$$

- Convert binary $(10110.001)_2$ to decimal.

Solution:

First, multiply each bit of $(10110.001)_2$ with its respective positional weight, and then we add the products of all the bits with its weight.

$$(10110.001)_2 = (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) + (0 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3})$$

$$(10110.001)_2 = (1 \times 16) + (0 \times 8) + (1 \times 4) + (1 \times 2) + (0 \times 1) + (0 \times 1/2) + (0 \times 1/4) + (1 \times 1/8)$$

$$(10110.001)_2 = 16 + 0 + 4 + 2 + 0 + 0 + 0 + 0.125$$

$$(10110.001)_2 = (22.125)_{10}$$

2.3 Decimal to Binary Conversion

For converting decimal to binary, there are two steps required to perform, which are as follows:

- i. Divide the given number by 2.
- ii. Get the integer quotient for the next iteration.
- iii. Get the remainder for the binary digit.
- iv. Repeat the steps until the quotient is equal to 0.

Examples:

- Convert $(13)_{10}$ to binary:

Solution:

Division by 2	Quotient	Remainder
13/2	6	1(LSB)
6/2	3	0
3/2	1	1
1/2	0	1(MSB)

So, $(13)_{10} = (1101)_2$

- Convert $(152.25)_{10}$ to binary

Solution:

Step 1: Divide the number 152 and its successive quotients with base 2.

Divide by 2	Quotient	Remainder
152/2	76	0 (LSB)
76/2	38	0
38/2	19	0
19/2	9	1
9/2	4	1
4/2	2	0
2/2	1	0
1/2	0	1(MSB)

$$(152)_{10} = (10011000)_2$$

Step 2: Now, perform the multiplication of 0.25 and successive fraction with base 2.

Operation	Result	carry
0.25×2	0.50	0
0.50×2	0	1

$$(0.25)_{10} = (.01)_2$$

So, $(152.25)_{10} = (10011000.01)_2$

2.4 Binary to Hexadecimal Conversion

The base numbers of binary and hexadecimal are 2 and 16, respectively. In a binary number, the pair of four bits is equal to one hexadecimal digit. There are also only two steps to convert a binary number into a hexadecimal number which are as follows:

- In the first step, we have to make the pairs of four bits on both sides of the binary point. If there will be one, two, or three bits left in a pair of four bits pair, we add the required number of zeros on extreme sides.
- In the second step, we write the hexadecimal digits corresponding to each pair.

Examples:

- Convert binary $(1101100)_2$ to hexadecimal.

Solution:

Convert every 4 binary bits (starting from LSB) to hex digit i.e.

$$1101100_2 = \overbrace{0110}^6 \overbrace{1100}^C = 6C = (6C)_{16}$$

Note: In the above example, the left side of the binary point has only three bits. To make it a complete pair of 4 bits, add one zero on the extreme side.

Table 2.2. Reference table for binary to hexadecimal equivalent

Binary () ₂	Hex Equivalent () ₁₆
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

➤ Convert binary **(10110101011.0011)₂** to hexadecimal.

Solution:

Firstly, make pairs of 4 bits on both sides of the binary point.

111 1010 1011.0011

On the left side of the binary point, the first pair has three bits. To make it a complete pair of four bits, add one zero on the extreme side i.e.

0111 1010 1011.0011

Now, write the hexadecimal digits correspond to each pair 4-bit pair i.e.

$$(0111\ 1010\ 1011.0011)_2 = (7AB.3)_{16}$$

2.5 Decimal to Hexadecimal Conversion

For converting decimal to hexadecimal, there are two steps required to perform, which are as follows:

- i. Divide the number by 16.
- ii. Get the integer quotient for the next iteration.
- iii. Get the remainder for the hex digit.
- iv. Repeat the steps until the quotient is equal to 0.

Examples:

- Convert $(7562)_{10}$ to hex

Solution:

Division by 16	Quotient (integer)	Remainder (decimal)	Remainder (hex)
7562/16	472	10	A
472/16	29	8	8
29/16	1	13	D
1/16	0	1	1

$$\text{So, } (7562)_{10} = (1D8A)_{16}$$

- Convert $(152.25)_{10}$ to hex

Solution:

First, divide the number 152 and its successive quotients with base 16.

Operation	Quotient	Remainder
152/16	9	8
9/16	0	9

$$(152)_{10} = (98)_{16}$$

Now, perform the multiplication of 0.25 and successive fraction with base 16.

Operation Result carry

0.25×16 0 4

$$(0.25)_{10} = (4)_{16}$$

Hence, the hexadecimal number of the decimal number $(152.25)_{10}$ is **$(230.4)_{16}$** .

2.6 Hexadecimal to Decimal Conversion

A regular decimal number is the sum of the digits multiplied with power of 10. For example, $(137)_{10}$ is equal to each digit multiplied with its corresponding power of 10, i.e.

$$(137)_{10} = 1 \times 10^2 + 3 \times 10^1 + 7 \times 10^0 = 100 + 30 + 7.$$

Likewise, hex numbers are read the same way, but each digit counts power of 16 instead of power of 10.

For hex number with n digits, i.e. $d_{n-1} \dots d_3 d_2 d_1 d_0$

Multiply each digit of the hex number with its corresponding power of 16 and sum:

$$\text{Hence, decimal} = d_{n-1} \times 16^{n-1} + \dots + d_3 \times 16^3 + d_2 \times 16^2 + d_1 \times 16^1 + d_0 \times 16^0$$

Examples:

➤ Convert $(E7A9)_{16}$ to decimal

Solution:

E7A9 in base 16 is equal to each digit multiplied with its corresponding 16^n i.e.

$$E7A9_{16} = 14 \times 16^3 + 7 \times 16^2 + 10 \times 16^1 + 9 \times 16^0 = 57344 + 1792 + 160 + 9 = 59305_{10}$$

Hence, **$(E7A9)_{16} = (59305)_{10}$**

➤ Convert $(3B)_{16}$ to decimal

Solution:

3B in base 16 is equal to each digit multiplied with its corresponding 16^n i.e.

$$3B_{16} = 3 \times 16^1 + 11 \times 16^0 = 48 + 11 = 59_{10}$$

Hence, **$(3B)_{16} = (59)_{10}$**

- Convert $(152A.25)_{16}$ to decimal

Solution:

First, multiply each digit of **152A.25** with its respective positional weight, and then we add the products of all the bits with its weight.

$$(152A.25)_{16} = (1 \times 16^3) + (5 \times 16^2) + (2 \times 16^1) + (A \times 16^0) + (2 \times 16^{-1}) + (5 \times 16^{-2})$$

$$(152A.25)_{16} = (1 \times 4096) + (5 \times 256) + (2 \times 16) + (10 \times 1) + (2 \times 16^{-1}) + (5 \times 16^{-2})$$

$$(152A.25)_{16} = 4096 + 1280 + 32 + 10 + (2 \times 1/16) + (5 \times 1/256)$$

$$(152A.25)_{16} = 5418 + 0.125 + 0.125$$

$$(152A.25)_{16} = 5418.14453125_{10}$$

Hence, $(152A.25)_{16} = (5418.14453125)_{10}$

2.7 Octal to Decimal Conversion

The process starts from multiplying the digits of octal numbers with its corresponding positional weights. And lastly, we add all those products.

Example:

- Convert $(152.25)_8$ to decimal

Solution:

First, multiply each digit of **152.25** with its respective positional weight, and then we add the products of all the bits with its weight.

$$(152.25)_8 = (1 \times 8^2) + (5 \times 8^1) + (2 \times 8^0) + (2 \times 8^{-1}) + (5 \times 8^{-2})$$

$$(152.25)_8 = 64 + 40 + 2 + (2 \times 1/8) + (5 \times 1/64)$$

$$(152.25)_8 = 64 + 40 + 2 + 0.25 + 0.078125$$

Hence, $(152.25)_8 = 106.328125$

3. Basic and Universal Logic Gates

3.1 Introduction

Logic gates are the basic building blocks of any digital system. It is an electronic circuit having one or more than one input and only one output. The relationship between the input and the output is based on certain logic. Based on this, logic gates are named as AND gate, OR gate, NOT gate etc. Moreover, NAND and NOR gates are called universal logic gates.

3.2 Logic Addition

The possible input and output combinations may be arranged as follows:

$$0 + 0 = 0$$

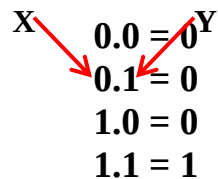
$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 1$$

3.3 Logical Multiplication

We can define the "." (logical multiplication) symbol or AND operator by listing all possible combinations for (input) variables let say X and Y and the resulting (output) value of X.Y will be as follows:

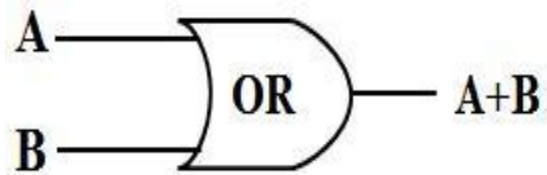

$$\begin{array}{l} 0.0 = 0 \\ 0.1 = 0 \\ 1.0 = 0 \\ 1.1 = 1 \end{array}$$

3.4 Logic Gates

- A logic gate is defined as an electronics circuit with two or more input signals and one output signal.
- AND, OR & NOT gates are called basic logic gates.
- NAND and NOR gates are called universal logic gates.

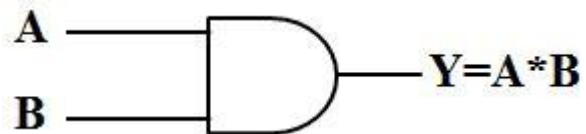
3.4.1 OR Gate

Inputs		Output
A	B	$Y = A + B$
0	0	0
0	1	1
1	0	1
1	1	1



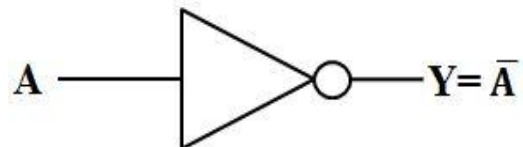
3.4.2 AND Gate

Inputs		Output
A	B	$Y = A * B$
0	0	0
0	1	0
1	0	0
1	1	1



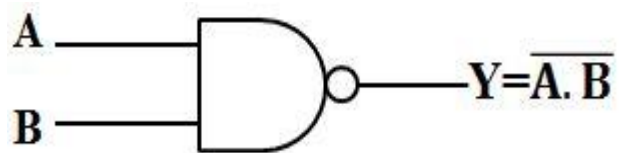
3.4.3 NOT Gate

Input	Output
A	Y
0	1
1	0



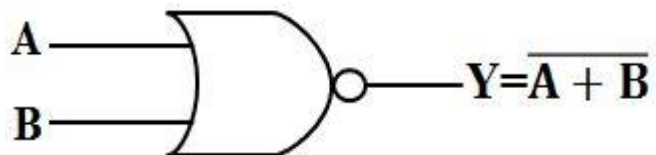
3.4.4 NAND Gate

Inputs		Output
A	B	$Y = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0



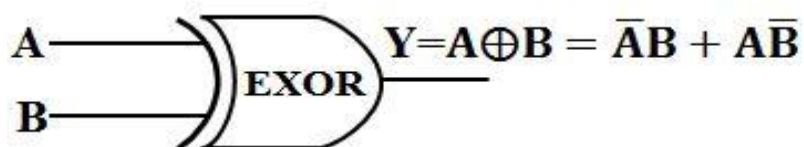
3.4.5 NOR Gate

Inputs		Output
A	B	$Y = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0



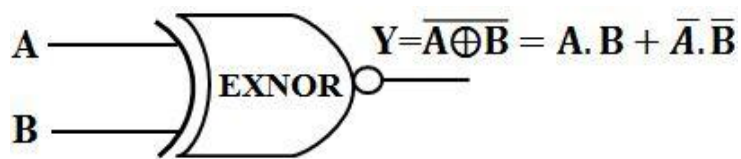
3.4.6 EX-OR Gate

Inputs		Output
A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0



3.4.7 EX-NOR Gate

Inputs		Output
A	B	$Y = A \oplus B$
0	0	1
0	1	0
1	0	0
1	1	1

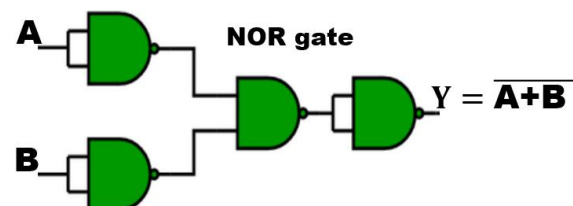
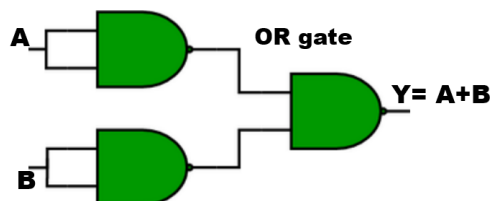
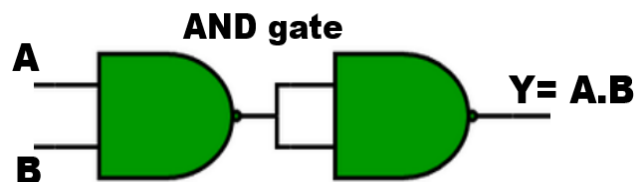
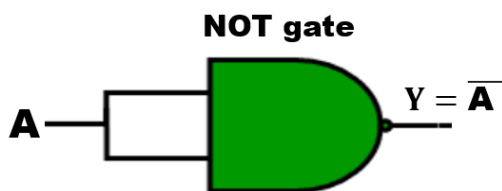


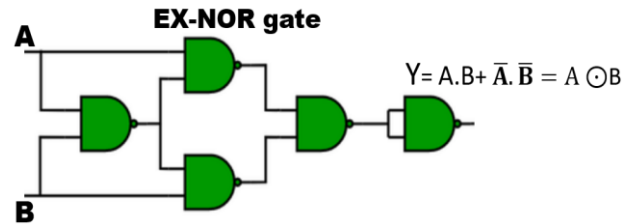
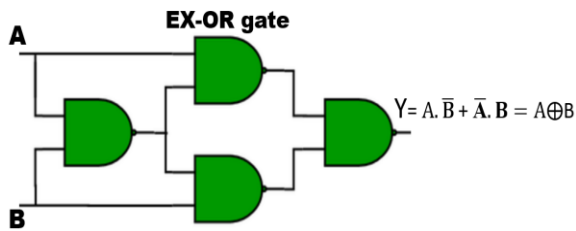
3.5 Realization of Logic Gate Using Universal gates

In Boolean Algebra, the **NAND** and **NOR** gates are called **universal gates** because any digital circuit can be implemented by using any one of these two i.e., any logic gate can be created using NAND or NOR gates only.

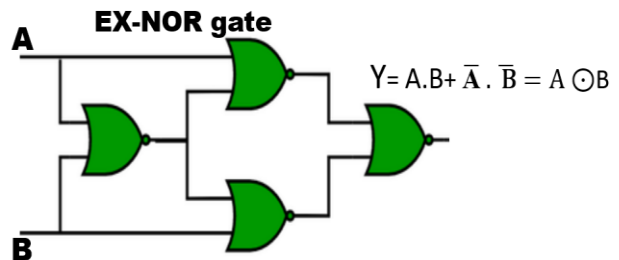
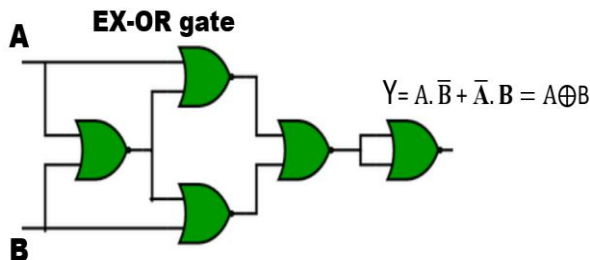
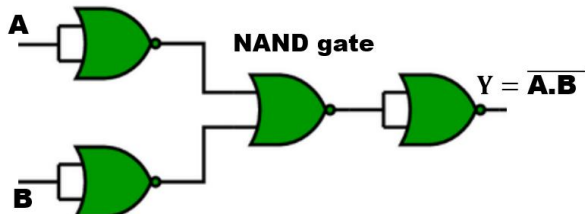
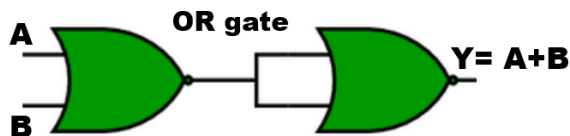
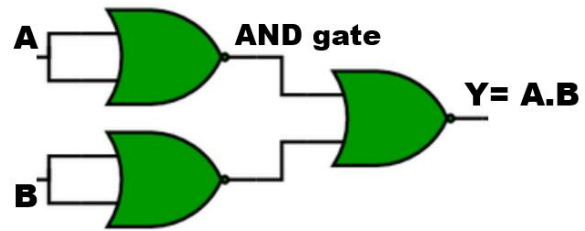
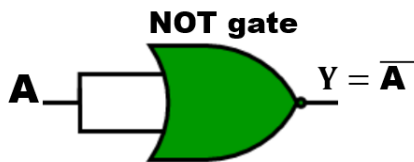
3.5.1 Implementation of NOT, AND, OR, NOR, NAND, EX-OR, & EX-NOR Gate Using Universal gates

A. Implementation Using Minimum Number of NAND Gates





B. Implementation Using Minimum Number of NOR Gates



4. Boolean Algebra Simplification of Boolean Function

4.1 Boolean Algebra

An algebra which deals with the binary number system called Boolean Algebra. Here, we use logic gate concept i.e.

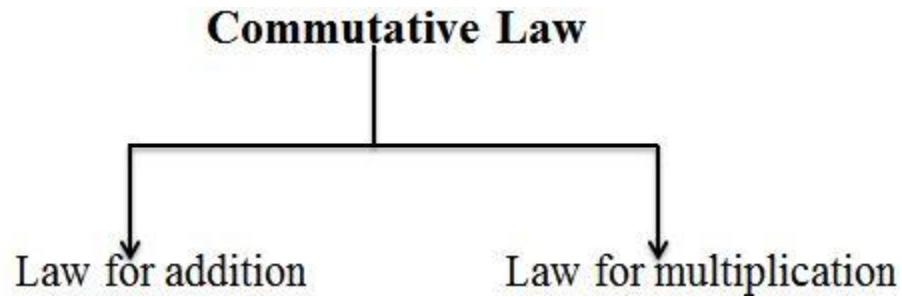
‘ + ’ Addition $\longrightarrow$ OR Gate

‘ • ’ Multiplication $\longrightarrow$ AND Gate

4.2 Laws of Boolean Algebra

- Commutative Law
- Associative Law
- Distributive Law

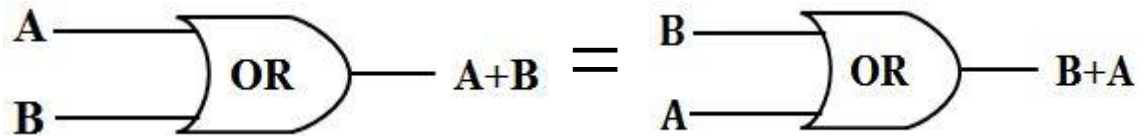
➤ Commutative Law



a) Law for addition

It is independent of order, this means if we change the sequence of variables, result will be same i.e.

$$A+B = B+A$$



b) Law for multiplication

It is also independent of order. It means changing variable sequence while AND operation, the final result will not affect i.e.

$$A.B = B.A$$

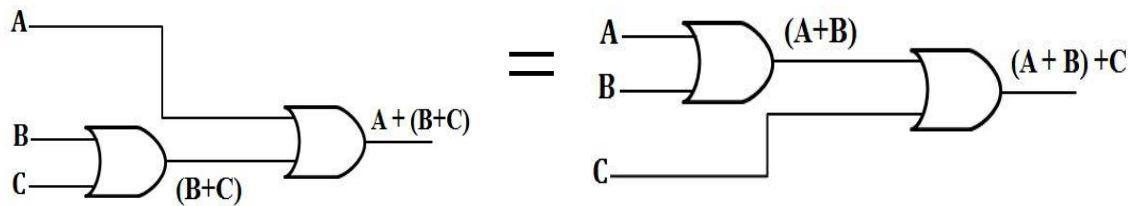


➤ Associative Law

a) Law for addition

Associative law for addition can be justified as follows:

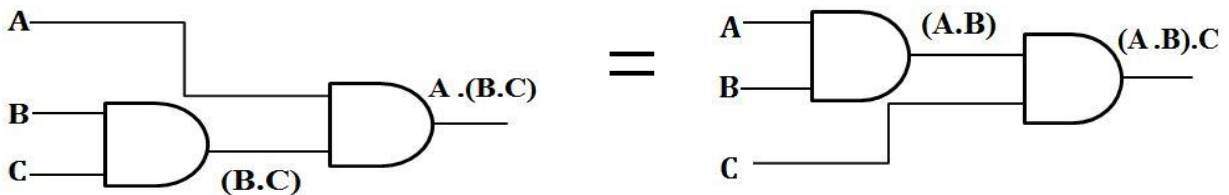
$$A + (B+C) = (A + B) + C$$



b) Law for multiplication

Associative law for multiplication can be justified as follows:

$$A \bullet (B \bullet C) = (A \bullet B) \bullet C$$

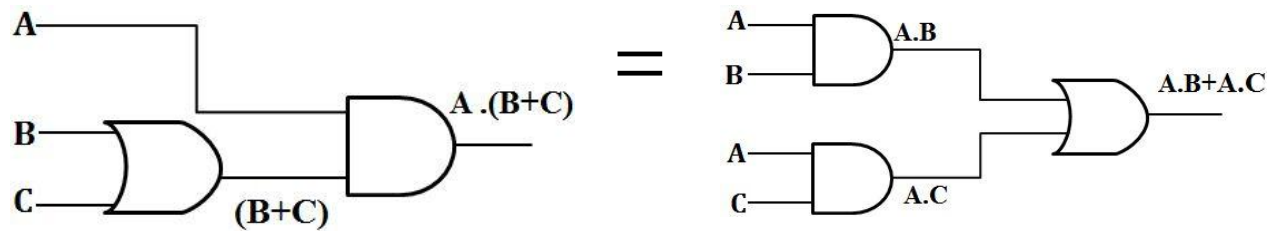


➤ Distributive Law

a) Law using AND operator

Distributive law using AND operator can be justified as follows:

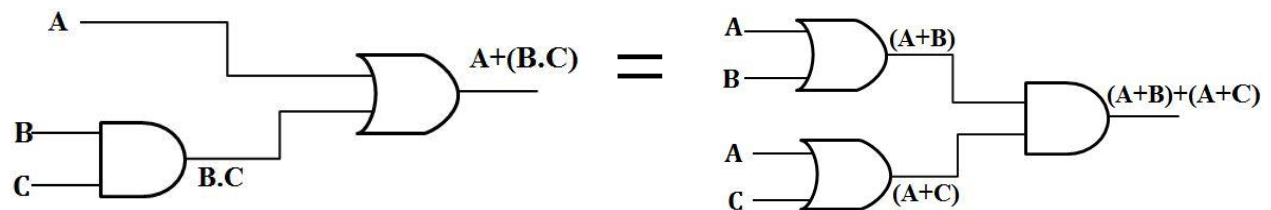
$$A \bullet (B+C) = A \bullet B + A \bullet C$$



b) Law using OR operator

Distributive law using OR operator can be justified as follows:

$$A + (B \bullet C) = (A+B) \bullet (A+C)$$



4.3 Duality in Boolean Algebra

Starting with a Boolean relation, we can derive another Boolean relation by-

- i. Changing each OR (+) sign to an AND (.) sign
- ii. Changing each AND (.) sign to an OR (+) sign
- iii. Complementary each 0 and 1

For example:

$$A + 0 = A$$

The dual relation is $A.1 = A$

Since, $A (B + C) = AB + AC$ (by distributive law).

Its dual relation is $A + B C = (A + B) (A + C)$

4.4 De Morgan's Theorem

This theorem can be justified as follows:

a) $\overline{A + B} = \bar{A} \cdot \bar{B}$

b) $\overline{A \cdot B} = \bar{A} + \bar{B}$

4.5 Boolean Algebra Theorems

Name	AND Form	OR Form
Identity Law	$A \cdot 1 = A$	$A + 0 = A$
Null Law	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent Law	$A \cdot A = A$	$A + A = A$
Inverse Law	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
Commutative Law	$AB = BA$	$A + B = B + A$
Associative Law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive Law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption Law	$A(A + B) = A$	$A + AB = A$
De Morgan's Law	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$

4.6 Simplification of Boolean expressions

In this section, we will discuss some working examples to simplify different Boolean expressions.

Example 1:

Simplify the Boolean expression $AB\bar{C}\bar{D} + AB\bar{C}D + ABC\bar{D} + ABCD$.

Solution:

Given, $AB\bar{C}\bar{D} + AB\bar{C}D + ABC\bar{D} + ABCD$

$$\Rightarrow AB\bar{C}(\bar{D} + D) + ABC(\bar{D} + D)$$

$$\Rightarrow AB\bar{C} \cdot 1 + ABC \cdot 1 \quad ; \quad (\text{As we know } D + \bar{D} = 1)$$

$$\Rightarrow AB\bar{C} + ABC$$

$$\Rightarrow AB(\bar{C} + C)$$

Hence, the answer is **AB**

Example 2:

Simplify the Boolean expression $AB(\bar{A} B \bar{C} + A \bar{B} \bar{C} + \bar{A} B C)$.

Solution:

Given, $AB(\bar{A} B \bar{C} + A \bar{B} \bar{C} + \bar{A} B C)$

$$\Rightarrow A \bar{A} B B \bar{C} + A A B \bar{B} \bar{C} + \bar{A} A B B C$$

$$\Rightarrow 0 + 0 + 0 \quad ; \quad (\text{As we know } A \bar{A} = 0)$$

Hence, the final answer is **0**

Example 3:

Simplify the Boolean expression $(\bar{A} \bar{B} + A B + \bar{A} B)$.

Solution:

Given, $(\bar{A} \bar{B} + A B + \bar{A} B)$

$$\Rightarrow \bar{A} \bar{B} + B(A + \bar{A})$$

$$\Rightarrow \bar{A} \bar{B} + B.1$$

$$\Rightarrow B + \bar{A} \bar{B}$$

$$\Rightarrow (B + \bar{A})(B + \bar{B})$$

$$\Rightarrow (B + \bar{A}).1$$

Hence, the answer is **$\bar{A} + B$**

Example 4:

Simplify the Boolean expression $(A + B)(A + \bar{B})$.

Solution:

$$\text{Given, } (A + B)(A + \bar{B})$$

$$\Rightarrow A.A + A.\bar{B} + A.B + B.\bar{B}$$

$$\Rightarrow A.A + A(\bar{B} + B) + 0$$

$$\Rightarrow A.A + A(\bar{B} + B)$$

$$\Rightarrow A + A$$

Hence, the answer is **A**

Example 5:

Simplify the Boolean expression $AB + A(CD + C\bar{D})$.

Solution:

$$\text{Given, } AB + A(CD + C\bar{D})$$

$$\Rightarrow AB + AC(D + \bar{D})$$

$$\Rightarrow AB + AC.(1)$$

Hence, the answer is **A(B + C)**

Example 6:

Simplify the Boolean expression $(B\bar{C} + \bar{A}D)(A\bar{B} + C\bar{D})$.

Solution:

$$\text{Given, } (B\bar{C} + \bar{A}D)(A\bar{B} + C\bar{D})$$

$$B\bar{C}A\bar{B} + B\bar{C}C\bar{D} + \bar{A}DA\bar{B} + \bar{A}DC\bar{D}$$

$$B\bar{B}\bar{C}A + \bar{C}CB\bar{D} + \bar{A}AD\bar{B} + \bar{A}CD\bar{D}$$

$$0 + 0 + 0 + 0 = 0$$

Hence, the answer is **0**

4.7 SOP and POS Representation for Logic Expressions

Sum of Products (SOP) form:-

Let the boolean function $f = AB + AC + BC$ (21)
It is in the form of sum of three terms AB, AC and BC with each individual term is product of two variables. therefore, such expressions are known as expression in SOP form. The sums and products in the SOP forms are not the actual addition or multiplication, in fact, they are the OR and AND functions.

Some more examples of SOP forms:-

(i) $F_1 = ABC + BCD + ABD$

(ii) $F_2 = xy + \bar{x}\bar{y} + x\bar{y}$

(iii) $F_2 = \bar{P}\bar{Q} + PQR + PQR$

Products of Sum (POS) form:-

Let the boolean function $F = (A+B) \cdot (B+C) \cdot (A+C)$
It is in the form of product of three terms $(A+B)$, $(B+C)$ and $(A+C)$ with each term is in the form of sum of two variables. products of Sum (POS) form

Some more examples of POS forms:-

$$F_1 = (A + \bar{B} + \bar{C}) \cdot (A + B) \cdot (\bar{A} + C)$$

$$F_2 = (\bar{x} + \bar{y}) \cdot (\bar{x} + y + z)$$

$$F_3 = (P + R) \cdot (P + \bar{Q}) \cdot (\bar{P} + R)$$

Sum of Products (SOP) form:-

(21)

Let the boolean function

$$F = AB + AC + BC$$

Sum
↑ ↑ ↑
products terms
Sum of Products (SOP) form

It is in the form of sum of three terms

AB, AC and BC with each individual term is product of two variables

therefore, such expressions are known as expression in SOP form.

The sums and products in the SOP forms are not the actual addition or multiplication. In fact, they are the OR and AND functions

Some more examples of SOP forms:-

(i) $F_1 = ABC + BCD + ABD$

(ii) $F_2 = xy + \bar{x}\bar{y} + x\bar{y}$

(iii) $F_2 = \bar{P}\bar{Q} + PQR + P\bar{Q}R$

Products of Sum (POS) form:-

Let the boolean function

It is in the form of product of three terms (A+B) (B+C) and (A+C) with each term

is in the form of sum of two variables.

Some more examples of POS forms:-

$$F_1 = (A + \bar{B} + \bar{C}) \cdot (A + B) \cdot (\bar{A} + C)$$

$$F_2 = (\bar{x} + \bar{y}) \cdot (\bar{x} + y + z)$$

$$F_3 = (P + R) \cdot (P + \bar{Q}) \cdot (\bar{P} + R)$$

$$F = (A+B) \cdot (B+C) \cdot (A+C)$$

products
↑ ↑ ↑
Sum terms
products of Sum (POS) form

Canonical or standard form:-

Standard SOP form $f = \underbrace{ABC}_{\uparrow} + \underbrace{A\bar{B}\bar{C}}_{\uparrow} + \underbrace{\bar{A}BC}_{\uparrow}$

Each product term consists of all the literals in the complemented or uncomplemented form

Standard POS form $f = \underbrace{(A+B+\bar{C})}_{\uparrow} \underbrace{(A+\bar{B}+C)}_{\uparrow} \underbrace{(\bar{A}+\bar{B}+\bar{C})}_{\uparrow}$

Each sum term consists of all the literals in the complemented or uncomplemented form.

Some more examples of standard or canonical SOP and POS form

- | Logical expression | Type of expression |
|---|--------------------|
| 1. $f = AB + ABC + A\bar{B}C$ | Non standard SOP |
| 2. $f = AB + A\bar{B} + \bar{A}B$ | Standard SOP |
| 3. $f = (\bar{A}+B)(A+B)(A+\bar{B})$ | Standard POS |
| 4. $f = (\bar{A}+B)(A+B+C)$ | Non standard POS |
| 5. $F(A,B,C) = ABC + \bar{A}\bar{B}\bar{C} + \bar{A}BC$ | Standard SOP |
| 6. $F(A,B,C) = BC + ABC + AB$ | Non standard SOP |

Minterm and Maxterm for three variables:-

			Minterm		Maxterm	
x	y	z	Term	Designation	Term	Designation
0	0	0	$\bar{x}\bar{y}\bar{z}$	m_0	$x+y+z$	M_0
0	0	1	$\bar{x}\bar{y}z$	m_1	$(x+y+\bar{z})$	M_1
0	1	0	$\bar{x}y\bar{z}$	m_2	$(x+\bar{y}+z)$	M_2
0	1	1	$\bar{x}yz$	m_3	$(x+\bar{y}+\bar{z})$	M_3
1	0	0	$x\bar{y}\bar{z}$	m_4	$(\bar{x}+y+z)$	M_4
1	0	1	$x\bar{y}z$	m_5	$(\bar{x}+y+\bar{z})$	M_5
1	1	0	$xy\bar{z}$	m_6	$(\bar{x}+\bar{y}+z)$	M_6
1	1	1	xyz	m_7	$(\bar{x}+\bar{y}+\bar{z})$	M_7

Ex: Express the given table in min term (SOP) and max term

(POS)

Decimal no	x	y	z	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	0
6	1	1	0	0
7	1	1	1	1

where output (F) is 1, these are min term (SOP)

$$F = \bar{x}\bar{y}z + x\bar{y}\bar{z} + xyz$$

or $F = m_1 + m_4 + m_7$

$$F = \sum m \{1, 4, 7\}$$

where output (F) is 0, these are max term (POS)

$$F = (x+y+z)(x+\bar{y}+\bar{z})(x+\bar{y}+z)(\bar{x}+y+\bar{z})(\bar{x}+y+z)$$

or

$$F = M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6$$

or $F = \prod M \{0, 2, 3, 5, 6\}$

Steps to convert Non standard form to standard form:-

1. Find missing literals in each terms.
2. And each product term having missing literals like $(x+\bar{x})$
3. Expand the terms by distributive law.
4. Write the repetitive terms only once.

Ex: Convert the given expression in standard form (SOP)

$$F(A, B, C) = AC + AB + BC$$

Steps.1 Missing literals $\downarrow \quad \downarrow \quad \downarrow$
 $B \quad C \quad A$

Step.2 $F(A, B, C) = AC(B+\bar{B}) + AB(C+\bar{C}) + BC(A+\bar{A})$
 $= ABC + A\bar{B}C + ABC + AB\bar{C} + ABC + \bar{A}BC$

$$F(A, B, C) = ABC + A\bar{B}C + AB\bar{C} + \bar{A}BC \quad \text{Standard (SOP)}$$

For pos form:-

Step 1 Find the missing literals

Step 2 Add that one with the required term as $x \cdot \bar{x}$

Step 3 Expand the expression.

Step 4 write the repetitive terms once.

Ex: convert the given expression into the standard pos form.

$$F(A, B, C) = (A+B) (B+C) (C+A)$$

missing literals $\uparrow \quad \uparrow \quad \uparrow$
 $C \quad A \quad B$

Add the missing literals $= (A+B+C\bar{C}) (B+C+A\bar{A}) (C+A+B\bar{B})$

$$F(A, B, C) = (A+B+C) \cdot (A+B+\bar{C}) \cdot (B+C+A) (B+C+\bar{A})$$

$$(C+A+B) (C+A+\bar{B})$$

$$F(A, B, C) = (A+B+C) (A+B+\bar{C}) (\bar{A}+B+C) (A+\bar{B}+C)$$

standard pos form.

conversion sop to pos and pos to sop:-

$$\text{sop } F = \sum_m \{0, 1, 2, 5\}$$

$$\text{pos } F = \prod_m \{3, 4, 6, 7\}$$

conversion from pos to sop:-

Ex: convert F_2 from pos to sop function.

$$F_2 = x(y+z)(\bar{x}+\bar{y}+\bar{z})$$

Take complement-

$$\bar{F}_2 = \overline{x(y+z)(\bar{x}+\bar{y}+\bar{z})}$$

$$\bar{F}_2 = \bar{x} + (y \cdot \bar{z}) + (x y z)$$

$$F_3 = \bar{F}_2 = \bar{x} + y\bar{z} + xyz \quad \text{any sop representation of } F_2$$

Ex: convert the function form pos to sop.

$$F = \prod_m \{0, 6, 7\}$$

Take complement $\bar{F} = [\prod_m \{0, 6, 7\}]'$

$$F = \sum_m \{1, 2, 3, 4, 5\} \quad \text{sop form.}$$

conversion from SOP to POS:-

ex: convert the following function from SOP to POS

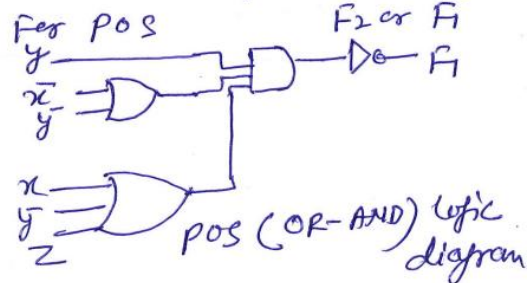
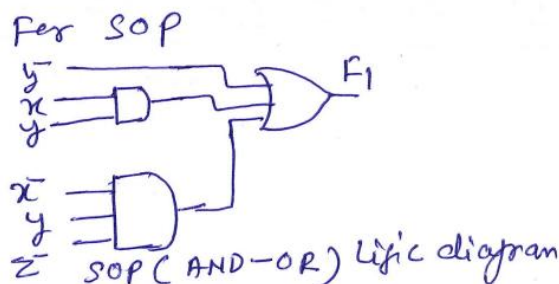
$$F_1 = \bar{y} + xy + \bar{x}y\bar{z}$$

take complement of F_1

$$F_2 = \bar{F}_1 = \overline{(\bar{y} + xy + \bar{x}y\bar{z})}$$

$$F_2 = \bar{F}_1 = y \cdot (\bar{x} + \bar{y}) \cdot (x + \bar{y} + z)$$

Ans
POS representation of F_1



5. Karnaugh map (K-Map)

5.1 Introduction

Karnaugh map or K-map is a map of a function used in a technique used for minimization or simplification of a Boolean expression. It results in less number of logic gates and inputs to be used during the fabrication.

5.2 Rules of Minimization in K-Map

- While grouping, you can make groups of 2^n number where $n=0, 1, 2, 3, \dots$
- You can either make groups of 1's or 0's but not both.
- Grouping of 1's lead to Sum of Product form and Grouping of 0's lead to product of Sum form.
- While grouping, the groups of 1's should not contain any 0 and the group of 0's should not contain any 1.'
- Groups can be made vertically and horizontally but not diagonally.
- Groups made should be as large as possible even if they overlap.
- All the like term should be in a group even if they overlap.

- Uppermost & lowermost squares can be made into a group together as they are adjacent (1-bit difference). Same goes for the corner squares.
- Each group represents a term in the Boolean expression. Larger the group, smaller and simple the term.
- The product of those literals that remains unchanged in a single group makes the term of the expression.
- Don't care "x" should also be included while grouping to make a larger possible group.

5.3 2-Variable K-Map

2 variables have $2^n = 2^2 = 4$ minterms. Therefore, there are 4 cells (squares) in 2 variable K-map for each minterms.

Consider A & B as two variables.

Grouping in 2 variables K-map is easy as there are few squares shown below

A \ B	0	1
0	m_0	m_1
1	m_2	m_3

Example of 2 Variable K-Map-

$$F = \sum (m_0, m_1, m_2) = \bar{A}\bar{B} + \bar{A}B + A\bar{B}$$

Now, K-map from Truth table

A \ B	0	1
0	m_0 1	m_1 1
1	m_2 1	m_3 0

Hence, the simplifies expression will be the sum of these two terms as given below,

$$F = \bar{A} + \bar{B}$$

5.4 3-Variable K-Map

3 variables make $2^n = 2^3 = 8$ min terms, so the K- map of 3 variables will have 8 squares(cells) as shown in the figure given below.

AB \ C	0	1
00	m ₀	m ₁
01	m ₂	m ₃
11	m ₆	m ₇
10	m ₄	m ₅

A \ BC	00	01	11	10
0	m ₀	m ₁	m ₃	m ₂
1	m ₄	m ₅	m ₇	m ₆

Moreover, we can make groups of 2, 4 & 8 cells having same 1s or 0s.

AB \ C	0	1
00	1	0
01	1	0
11	0	1
10	0	1

AB \ C	0	1
00	1	0
01	0	0
11	0	0
10	1	0

AB \ C	0	1
00	0	0
01	1	1
11	1	1
10	0	0

AB \ C	0	1
00	1	0
01	1	0
11	1	0
10	1	0

Example of 3 Variable K-Map:

Simplify the given function using K-map.

$$F(A, B, C) = \sum (m_0, m_2, m_3, m_4, m_6, m_7)$$

Solution:

		C	
		0	1
AB	00	m ₀ 1	m ₁ 0
	01	m ₂ 1	m ₃ 1
	11	m ₆ 1	m ₇ 1
	10	m ₄ 1	m ₅ 0

The sum of these two terms will make the simplified expression of the function as given below.

$$F = B + \bar{C}$$

5.5 4-Variable K-Map

Four variables have $2^n = 2^4 = 16$ minterms. So, a 4-variable k-map will have 16 cells as shown in the figure given below.

		CD			
		00	01	11	10
AB	00	m ₀	m ₁	m ₃	m ₂
	01	m ₄	m ₅	m ₇	m ₆
	11	m ₁₂	m ₁₃	m ₁₅	m ₁₄
	10	m ₈	m ₉	m ₁₁	m ₁₀

Each cell (min term) represents the variables in front of the corresponding row & column.

Some examples of grouping in 4-variable k-map are given below:

CD \ AB	00	01	11	10
00	1	0	0	0
01	1	1	1	1
11	1	1	1	0
10	1	0	0	0

CD \ AB	00	01	11	10
00	0	1	1	0
01	1	0	0	1
11	1	0	0	1
10	0	1	1	0

Example of 4 Variable K-Map:

Simplify the given function using K-map.

$$F(A, B, C, D) = \sum (m_0, m_1, m_4, m_5, m_6, m_8, m_9, m_{12}, m_{13})$$

Solution:

CD \ AB	00	01	11	10
00	1	1	0	0
01	1	1	0	1
11	1	1	0	0
10	1	1	0	0

Group of 8 will give a term of 1 literal that remains unchanged i.e. $\bar{C}$

So, the expression will $F = \bar{C} + \bar{A}B\bar{D}$

6. Binary Arithmetic:

Binary arithmetic: $\rightarrow$ computer circuits do process decimal numbers; they process binary numbers. let us discuss binary arithmetic. Binary addition is the key to binary subtraction, multiplication and division. Binary arithmetic is much simpler than decimal arithmetic because here only, 0 and 1 are involved.

Binary addition:-

A	B	sum	carry
0 + 0		0	0
0 + 1		1	0
1 + 0		1	0
1 + 1		0	1

Ex'
$$\begin{array}{r} 1010 \\ + 0011 \\ \hline 1101 \end{array}$$

$$\begin{array}{r} 10 \\ + 3 \\ \hline 13 \end{array}$$

Ex'
$$\begin{array}{r} 11100 \\ + 01111 \\ \hline 101011 \end{array}$$

$$\begin{array}{r} 28 \\ + 15 \\ \hline 43 \end{array}$$

Ex'
$$\begin{array}{r} 01101010 \\ 00001000 \\ 10000001 \\ + 11111111 \\ \hline 111110010 \end{array}$$

Binary Subtraction:-

A	B	Diff.	Borrow
0 - 0		0	0
0 - 1		1	1
1 - 0		1	0
1 - 1		0	0

Ex'
$$\begin{array}{r} 1011 \\ - 0101 \\ \hline 0110 \end{array}$$

$$\begin{array}{r} 11 \\ - 5 \\ \hline 6 \end{array}$$

Ex'
$$\begin{array}{r} 1011 \\ - 0110 \\ \hline 0101 \end{array}$$

$$\begin{array}{r} 11 \\ - 6 \\ \hline 5 \end{array}$$

Binary multiplication:-

$$\begin{array}{r} 1001 \\ \times 1101 \\ \hline 1001 \\ 0000 \\ 1001 \\ 1001 \\ \hline 1110101 \end{array}$$
 Ans

Binary division:— Binary division is obtained using the same procedure as decimal division. (10)

Ex: Divide 1110101 by 1001

$$\begin{array}{r}
 1101 \\
 1001 \overline{) 1110101} \\
 \underline{-1001} \\
 1011 \\
 \underline{-1001} \\
 1001 \\
 \underline{1001} \\
 0000
 \end{array}$$

Ans = 1101

7. Complements

One's (1's) complement Representation:— ☺

In a binary no, if each one (1) is replaced by 0 and each 0 by 1, the resulting no is known as the 1's complement of the first no. In fact, both the numbers are complement of each other. If one of these no is +ve, then the other number will be -ve with the same magnitude and vice versa.

Ex: $(0101)_2 \rightarrow (+5)_{10}$
 $(1010)_2 \rightarrow (-5)_{10}$

This method is widely used for representing signed numbers. In this representation also, MSB is 0 for +ve no and 1 for -ve numbers.

Ex: $(0100111001)_2 \rightarrow (1011000110)_{(1's \text{ comp})}$ (b) $\begin{cases} 11011010 - \text{binary} \\ 000100101 - 1's \text{ comp.} \end{cases}$

2's complement representation:-

(12)

If 1 is added to 1's complement of a binary no., the resulting no is known as 2's complement of the binary no.

Ex: $(0101)_2 \rightarrow 1011$ (2's comp)

Since (0101) represent $(+5)_{10}$ therefore (1011) represent $(-5)_{10}$ in 2's complement representation. if the MSB is 0 the number is +ve, whereas if the MSB is 1 the number is -ve.

Ex: Find 2's complement-

$$\begin{array}{r} \text{(i)} \quad 01001110 \\ \text{1's comp} \quad 10110001 \\ \quad \quad \quad + 1 \\ \hline \text{2's comp} \quad 10110010 \end{array}$$

$$\begin{array}{r} \text{(ii)} \quad 00110101 \\ \text{1's comp} \quad 11001010 \\ \quad \quad \quad + 1 \\ \hline \text{2's comp} \quad 11001011 \end{array}$$

7.1 Binary Subtraction using Complements (1s & 2s)

Binary Subtraction using 1's & 2's complement

1's complement :- To calculate the 1's complement of a binary number just flip each value (bit) of the original binary no.
i.e. $0 \rightarrow 1$, $1 \rightarrow 0$

Ex 0 1 0 1 1 1
 ↓ 1's complement
 1 0 1 0 0 0

2's complement :- Just add '1' to 1's complement of that number.

Ex 010111 $\xrightarrow{2's \text{ Complement}}$ 1's Complement + 1
 101000 + 1
 ↓
 101001
 (2's complement).

NOTE :-

1's & 2's Complement are taken for only -ve numbers.

SUBTRACTION USING 1'S COMPLEMENT

Ex:- Perform $(9)_{10} - (4)_{10}$ using 1's complement.

$\downarrow$

$\begin{matrix} 8 & 4 & 2 & 1 \\ (1 & 0 & 0 & 1) \end{matrix} \begin{matrix} 8 & 4 & 2 & 1 \\ (0 & 1 & 0 & 0) \end{matrix} \begin{matrix} 2 \\ 2 \end{matrix}$

[AKTU QUESTION].

$$\begin{array}{r} 9 - 4 \\ = 5 \end{array}$$

Step 1: Take 1's complement of -ve number i.e. (0100).

↓
1011

Step 2: Then Add first number with 1's complement of second no.

$$\begin{array}{r} 1001 \\ 1011 \\ \hline 0100 \end{array}$$

final carry. ← 1

Remember	
0+0	⇒ 0
0+1	= 1
1+0	= 1
1+1	= 1 0

Carry.

Step 3: Add final carry to the above result-

$$\begin{array}{r} 0100 \\ +1 \\ \hline (0101)_2 \end{array}$$

Ans. Answer is positive & True form.

Ex - Perform $(1011001)_2 - (1101010)_2$ using 1's complement

Step 1: (1101010)
↓ 1's complement
0010101

89-106
= 17

Step 2: Add both number

$$\begin{array}{r} 1011001 \\ 0010101 \\ \hline 1101110 \end{array}$$

final carry. 0

→ Answer is -ve & 1's complement form

Step 3: Invert all the bits

$$\begin{array}{r} 1101110 \\ \downarrow \\ (0010001)_2 \end{array}$$

Ans

SUBTRACTION USING 2'S COMPLEMENT

Ex: Perform following subtraction using 2's complement method.

$$(1101)_2 - (1001)_2$$

$$13 - 9 = 4$$

Step 1: Take 2's complement of -ve number (1001)

$$\downarrow$$
$$0110 \text{ (1's complement)}$$

$$\downarrow$$
$$0110 + 1 \text{ (Add 1)}$$

$$\downarrow$$
$$\text{2's complement: } (0111)_2$$

Step 2: Add both number $(1101)_2 + (0111)_2$

$$\begin{array}{r} 1101 \\ 0111 \\ \hline 10100 \end{array}$$

final carry $\leftarrow$ 1 0100 $\rightarrow$ Ans is +ve & True form. = 4

Ex Perform binary subtraction using 2's complement method:-

$$(10011)_2 - (1101)_2$$

$$19 - 13 = 6$$

1. $(01101)_2 \rightarrow$ 2's complement

$\downarrow$

$$10010 \text{ (1's complement)}$$

$\downarrow$

$$10010 + 1 \rightarrow 10011 \text{ (add 1)}$$

2. Add both the no.

$$\begin{array}{r} 10011 \\ + 10011 \\ \hline \end{array}$$

final carry $\leftarrow$ 1 00110 $\rightarrow$ Ans is +ve (6) & true form.

(Make both number of equal digits)

EX Perform binary subtraction using 2's complement method -
 $(011100)_2 - (101010)_2$ $(28 - 42) = 14$

1. $(101010)_2 \longrightarrow 2's \text{ complement}$

$\downarrow$
 010101

$\downarrow$
 $010101 + 1 = 010110$

2. Add

$\begin{array}{r} 011100 \\ + 010110 \\ \hline \end{array}$

final carry $\boxed{0} \boxed{110010} \rightarrow$

Ans is -ve & its 2's complement form.

3. Invert + 1 : 001101 (1's complement)
 all bits

$+1 \downarrow$

$\boxed{001110}$ 2's complement

(14)

Ans

NOTE: final carry

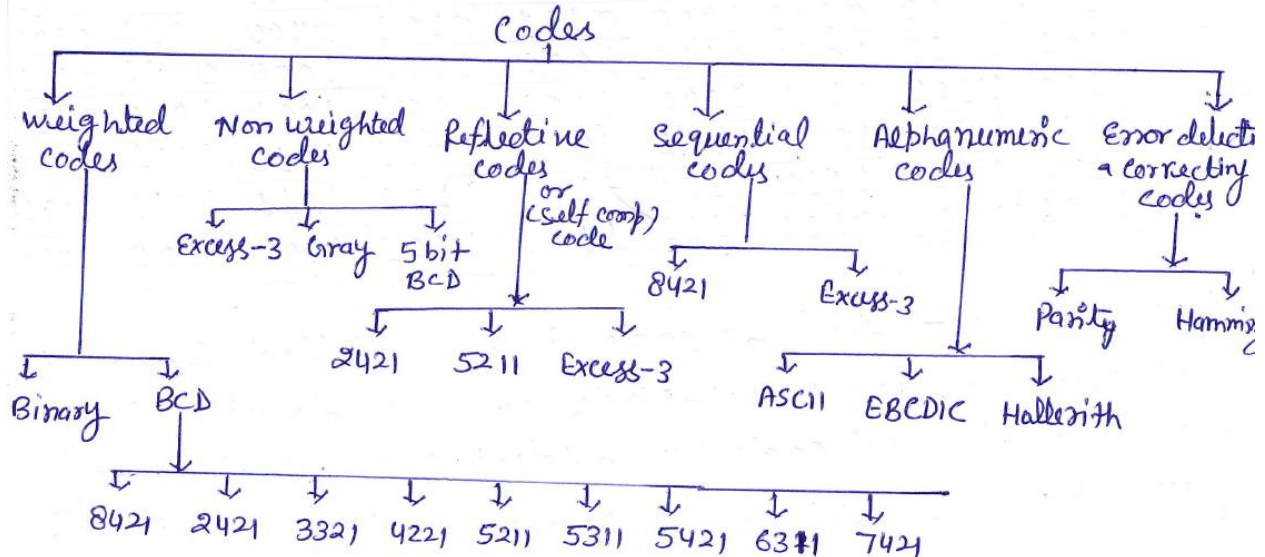
1 : Answer is +ve & true form.

0 : Answer is -ve & 2's complement form.

8. Codes (Weighted and Non-weighted codes, BCD & Gray Code)

Binary code: - The digital data is represented, stored⁽¹³⁾ and transmitted as groups of binary digits (bits). The group of bits, also known as binary codes.

classification of codes: -



Binary coded decimal (BCD): -

* In BCD each decimal digit is represented by a 4 bit binary number.

Decimal symbol

BCD digit

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	00010000
11	00010001
12	00010010
13	00010011
14	00010100
15	00010101

Ex (185)₁₀ → ()_{BCD}

= (0001 1000 0101)_{BCD}

but = (1011001)₂

BCD: code:-

(14)

Decimal	8-4-2-1 BCD	2+4+2+1 BCD	3+3+2+1 BCD	4+2+2+1 BCD	5 2 2 1 BCD	5 3 1 1 BCD	5 4 2 1 BCD	6 3 1 1 BCD	7 4 2 1 BCD
0	0000	0000	0000	0000	0000	0000	0000	0000	0000
1	0001	0001	0001	0001	0001	0001	0001	0001	0001
2	0010	0010	0010	0010	0011	0011	0010	0011	0010
3	0011	0011	0011	0011	0101	0100	0011	0100	0011
4	0100	0100	0101	1000	0111	0101	0100	0101	0100
5	0101	1011	1010	0111	1000	1000	1000	0111	0101
6	0110	1100	1100	1100	1001	1001	1001	1000	0110
7	0111	1101	1101	1101	1011	1011	1010	1001	1000
8	1000	1110	1110	1110	1101	1100	1011	1011	1001
9	1001	1111	1111	1111	1111	1101	1100	1100	1010

Excess-3 code:- Excess-3 code is a modified form of a BCD number. The Excess-3 code can be derived from the natural BCD code by adding 3 to each coded number.

Decimal no	Excess-3 code	BCD no.
0	0011	0000
1	0100	0001
2	0101	0010
3	0110	0011
4	0111	0100
5	1000	0101
6	1001	0110
7	1010	0111
8	1011	1000
9	1100	1001

Ex' $(592)_{10} \rightarrow (1000 \ 1100 \ 0101)_{\text{Excess-3 code}}$
 $(403)_{10} \rightarrow (0111 \ 0011 \ 0110)_{\text{Excess-3 code}}$

Ex: (i) Add two Excess-3 codes
 (ii) If carry 1 $\rightarrow$ add 3 to the sum of two digits.
 0 $\rightarrow$ Subtract 3

Ex: $1011 \rightarrow$ Excess-3 for 8
 Carry $\underline{1} + 1001 \rightarrow$ Excess-3 for 6

$$\begin{array}{r} 1011 \\ + 1001 \\ \hline 0100 \end{array}$$

 (3) $\left[\begin{array}{r} 8 \\ +6 \\ \hline 14 \end{array} \right]$
 (4) $\left[\begin{array}{r} 8 \\ +6 \\ \hline 14 \end{array} \right]$

Gray code: - (Unit distance code): -

Gray code is a special case of unit distance code. In unit distance code, bit patterns for two consecutive numbers differs in only 1 bit position. These codes are also called cyclic codes.

Decimal code	Gray code	Binary no.
0	0000	0000
1	0001	0001
2	0011	0010
3	0010	0011
4	0110	0100
5	0111	0101
6	0101	0110
7	0100	0111
8	1100	1000
9	1101	1001
10	1111	1010
11	1110	1011
12	1010	1100
13	1011	1101
14	1001	1110
15	1000	1111

Binary to Gray conversion:-

Let us represent binary no as

$B_1 B_2 B_3 B_4 \dots B_n$ and its equivalent Gray code as
 $G_1 G_2 G_3 G_4 \dots G_n$

with this representation, Gray code bits are obtained from the binary bits as under:

[MSB $\rightarrow$ As it is]
or same

$$G_1 = B_1$$

$$G_2 = B_1 \oplus B_2 \text{ (Ex-or)}$$

$$G_3 = B_2 \oplus B_3 \text{ (")}$$

$$G_4 = B_3 \oplus B_4$$

$$\vdots$$
$$G_n = B_{n-1} \oplus B_n$$

Ex. Convert $(10111011)_2 \rightarrow$ Gray code

$$\begin{array}{cccccccc} \text{Binary code} & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ \text{Gray code} & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{array}$$

(Ex-or'ing)

Ex-or table	
A	B
0	0
0	1
1	0
1	1

So, $(10111011)_2 \rightarrow (11100110)$ Gray code Ans

Gray to Binary conversion:-

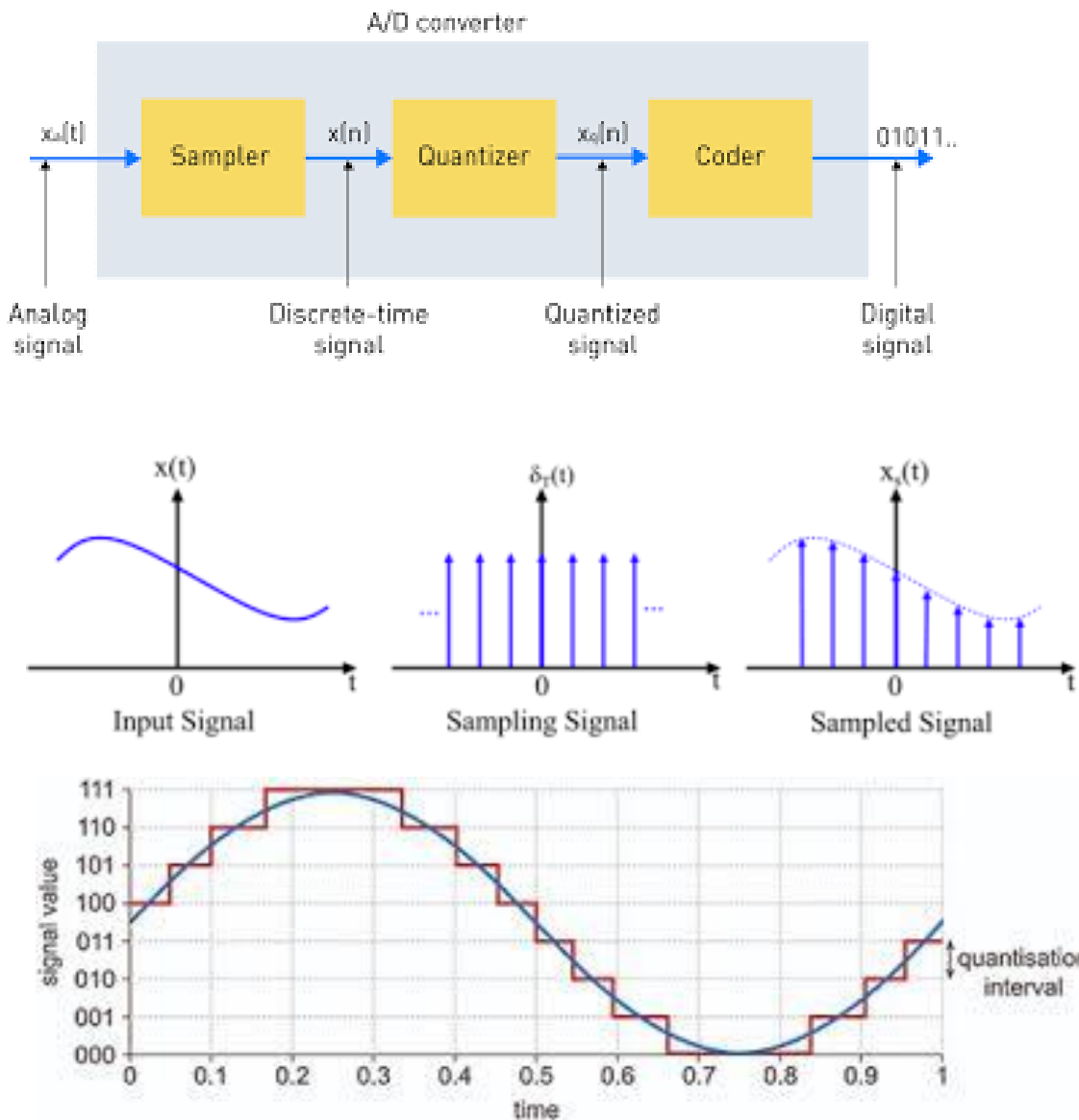
- (i) The MSB of the binary number is the same as the MSB of the Gray code number. So, write it down.
- (ii) To obtain the next binary no, perform an Ex-OR operation between the bit just written down and the next gray code bit, write down the result.
- (iii) Repeat step-2 until all gray code bits have been Ex-ORed with binary digits.

Ex. Convert gray code 101011 into its binary equivalent

$$\begin{array}{ccccccc} \text{Gray code} & 1 & 0 & 1 & 0 & 1 & 1 \\ & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ \text{Binary code} & 1 & 1 & 0 & 0 & 1 & 0 \end{array}$$

So, $(101011)_{\text{gray code}} \rightarrow (110010)_2$ Ans

9. Sampling & Quantization in Digital Signal Processing



Sampling is the process of converting a continuous signal's domain (like time or space) into a discrete sequence of points, while quantization is the subsequent process of converting the signal's amplitude at those discrete points into a limited, finite set of digital values. Together, these two steps, performed by an analog-to-digital converter (ADC), transform analog signals into the discrete digital format required for digital processing, storage, and transmission.

9.1. Sampling

- Definition:

Sampling is the process of discretizing the domain of a continuous signal, such as taking measurements of a signal's amplitude at regular intervals in time or space.

- Purpose:

It allows for the representation of a complex, continuous signal as a sequence of numerical values, making it suitable for digital processing.

9.2. Quantization

- Definition:

Quantization is the process of converting the continuous amplitude values obtained from sampling into a finite number of discrete, predefined values or levels.

- Purpose:

It digitizes the range or amplitude of the signal, replacing the precise sampled real number with the closest value from a limited set of discrete choices.

In Summary

- Sampling: digitizes the signal's coordinate values (e.g., time, x-axis).
- Quantization: digitizes the signal's amplitude values (e.g., magnitude, y-axis).
- Both processes are essential for converting an analog signal into a digital one, with the accuracy and quality of the final digital signal depending on the sampling rate and the number of quantization levels used

10. Fundamentals and Applications of Quantum Computing and Edge Computing

10.1. Quantum Computing

Quantum computing is a type of computing that uses the principles of quantum mechanics, such as superposition, entanglement, and interference, to process information in a fundamentally different from classical computers.

Its basic unit of information is the qubit, which can represent both 0 and 1 simultaneously, enabling parallel computations that allow quantum computers to solve certain complex problems exponentially faster than classical computers.

Core Fundamentals:

- **Superposition:**

Unlike classical bits that are either 0 or 1, a qubit can be in a superposition of both states simultaneously. This allows a quantum computer to explore multiple possibilities at once.

- **Entanglement:**

This quantum phenomenon links two or more qubits so that their states are correlated, even when physically separated. Measuring the state of one entangled qubit instantly reveals information about the other, no matter the distance.

- **Interference:**

Quantum interference is used to manipulate the probability of different outcomes, reinforcing correct answers and cancelling out incorrect ones in a quantum computation.

Key Components

- **Qubits:**

The fundamental unit of quantum information. A system of k qubits can represent 2^k distinct states, providing exponential parallel processing power compared to k classical bits.

- **State Vector:**

The mathematical description of a quantum system's state, a vector where each component's squared magnitude represents the probability of observing a particular output.

- **Measurement:**

A quantum computation ends with measurement, which collapses the superposition state into a single classical outcome (a sequence of bits), chosen probabilistically according to the quantum state.

Quantum vs. Classical Computing

- **Speed:**

For certain complex problems, quantum computers offer a significant speed advantage, such as finding a secret key with $O(\sqrt{n})$ complexity compared to a classical computer's $O(n)$ complexity.

- **Applications:**

Quantum computers are ideal for tasks like simulating complex physical systems (e.g., molecular behavior, protein folding), which are beyond the reach of even the most powerful classical supercomputers.

Applications of Quantum Computing

1. Cryptography

- Breaking classical encryption (via Shor's algorithm).
- Developing quantum-resistant algorithms (post-quantum cryptography).
- Quantum key distribution (QKD) for secure communication.

2. Quantum Simulation

- Simulating quantum systems (e.g., molecules, materials) that are intractable for classical computers.
- Applications in drug discovery, chemistry, and materials science.

3. Optimization Problems

- Solving complex combinatorial problems in logistics, finance, and operations research.
- Algorithms like Quantum Approximate Optimization Algorithm (QAOA) show promise.

4. Machine Learning

- Quantum machine learning (QML) models may speed up data analysis, classification, and pattern recognition.
- Algorithms like quantum support vector machines, quantum neural networks, etc.

5. Financial Modeling

- Risk analysis, portfolio optimization, and Monte Carlo simulations.
- Potential to improve speed and accuracy of financial predictions.

6. Artificial Intelligence

- Quantum-enhanced data processing and decision-making.
- Currently more theoretical, but research is advancing.

10.2. Edge Computing

Edge computing brings data processing and storage closer to the data's source, rather than sending all data to a central cloud or data center. This distributed architecture reduces latency, conserves bandwidth, enhances reliability by minimizing network dependency, and improves data privacy.

The fundamentals involve a layered approach, with devices generating data at the bottom layer, edge devices processing and reducing data at the middle layer, and the cloud for long-term storage and big data analysis at the top layer.

Core Principles

- **Proximity:**

The key principle is to perform computation as close as possible to the data source, at the "edge" of the network.

- **Decentralized Processing:**

Instead of a single, centralized cloud, computing resources are distributed across numerous edge devices and local gateways.

- **Local Data Handling:**

Data is processed, filtered, and analyzed at the edge, with only essential results or summaries sent to the cloud.

Benefits

- **Lower Latency:**

Reduced distance between data sources and processing centers leads to faster response times for time-sensitive applications.

- **Reduced Bandwidth Costs:**

By processing and filtering data locally, the volume of data sent over the network to the cloud is significantly reduced, saving bandwidth and costs.

- **Increased Reliability:**

Applications can continue to operate even with intermittent or limited network connectivity, as processing occurs locally.

- **Enhanced Security and Privacy:**

Sensitive data can be processed and anonymized at the edge, reducing the risk of exposure during transmission to the cloud.

- **Improved Performance:**

Local processing allows for real-time insights and faster, data-driven decision-making.

Architecture

An edge computing architecture typically includes:

- **Device Layer:**

End-user devices, smartphones, IoT sensors, and computers that generate data.

- **Edge Layer:**

Edge gateways and small servers that perform local data processing, filtering, and caching.

- **Cloud Layer:**

Centralized servers for large-scale data storage, complex analysis, and long-term data warehousing.

Applications of Edge Computing

1. Internet of Things (IoT)

- Local processing in smart sensors and devices.
- Reduces the need for constant cloud connectivity.

Examples:

- Smart thermostats
- Industrial IoT (IIoT) in factories

2. Autonomous Vehicles

- Real-time data processing from cameras, LIDAR, and radar.
- Decisions (e.g., braking, obstacle avoidance) must happen in milliseconds.

3. Healthcare

- Edge-enabled medical devices for real-time monitoring (e.g., wearables).
- Faster decision-making in critical scenarios like ICU or emergency rooms.

4. Retail

- Smart shelves and cameras analyze customer behavior in real-time.
- Edge systems can manage inventory, digital signage, and POS systems.

5. Smart Cities

- Edge computing supports traffic management, surveillance, and environmental monitoring.
- Enables quick decision-making based on real-time data.

6. Manufacturing (Industry 4.0)

- Predictive maintenance using local data analytics.
- Quality control and machine optimization at the edge.

7. Augmented Reality (AR) / Virtual Reality (VR)

- Real-time rendering for immersive experiences.
- Used in gaming, remote collaboration, and training.

8. Content Delivery & Streaming

- Edge servers cache and deliver content locally to reduce latency.
- Improves user experience for video streaming, gaming, etc.

Edge computing is essential for enabling low-latency, high-efficiency computing in environments where cloud dependence is impractical or inefficient. Its synergy with **IoT**, **AI/ML**, and **5G** makes it foundational for the future of connected and intelligent systems.

11. Summary of the Unit

- This unit introduces the principles and applications of digital electronics, covering key topics such as number systems, binary arithmetic, Boolean algebra, logic gates, and combinational and sequential circuits.
- These concepts are essential for understanding the operation and design of digital devices. The unit begins with an exploration of various number systems, including binary, octal, and hexadecimal, and their conversions, which are crucial for digital computation. It then delves into binary arithmetic operations such as addition, subtraction, multiplication, and division.
- Boolean algebra is introduced as the mathematical foundation for digital logic, along with theorems and properties that facilitate the simplification of logical expressions. The unit also examines logic gates, including AND, OR, NOT, NAND, NOR, XOR, and XNOR gates, and their truth tables.
- It also discusses about the simplification of Boolean Expressions using K-Map (upto 4 variables) and the implementation of the solution using NAND and NOR gates.
- This unit also has a brief introduction to damping and quantization in digital signal processing i.e. the conversion from analog to digital signal.
- Moreover, it also focusses on the latest technologies of Edge computing and Cloud computing.

12. Previous Year University Questions

12.1. Short Questions with Answer

1. What is a Logic gate?

Answer: A logic gate is a basic building block of digital circuits that performs a specific logical operation (like AND, OR, NOT) on one or more binary inputs to produce a single binary output.

2. What is DeMorgan's theorem?

Answer: DeMorgan's Theorem provides two transformation rules for simplifying Boolean expressions:

- i. $(A \cdot B)' = A' + B'$; The complement of a product is equal to the sum of the complements.
- ii. $(A + B)' = A' \cdot B'$; The complement of a sum is equal to the product of the complements.

3. Define minterms and maxterms.

Answer:

Minterm: A minterm is a standard product (AND) term in Boolean algebra where each variable appears exactly once, either in its true or complemented form. It corresponds to a single row in the truth table where the function is 1.

Maxterm: A maxterm is a standard sum (OR) term in Boolean algebra where each variable appears exactly once, either in its true or complemented form. It corresponds to a single row in the truth table where the function is 0.

4. What are universal gates? Why are they called so?

Answer: Universal gates are logic gates (NAND and NOR) that can be used to implement any other basic gate (AND, OR, NOT) and, therefore, any digital circuit.

They are called “universal” because their versatility allows the construction of any logic function using only one type of gate.

5. What is the use of don't care combinations? Is it an advantage or disadvantage to include them in a map?

Answer: Don't care combinations are input conditions in a truth table where the output does not affect the circuit's functionality. They are used in Karnaugh maps (K-maps) to simplify Boolean expressions by treating them as either 0 or 1, whichever minimizes the circuit. Moreover, Including don't care combinations helps in optimizing the circuit, reducing the number of gates and complexity.

6. Distinguish between SOP and POS.

Answer: SOP (Sum of Products) and POS (Product of Sums) are two standard forms of representing Boolean expressions.

In SOP, the expression is written as a sum (OR) of product (AND) terms, derived from truth table rows where the output is 1

Example: $F = AB + A'CF = AB + A'C$

POS represents the expression as a product (AND) of sum (OR) terms, derived from rows where the output is 0

Example: $F = (A + B)(A' + C)$

12.2. Previous Year Numericals with Solution (AKTU)

1. Convert the following to the corresponding bases [AKTU, 2020-21, 2021-22]

i. $(73.12)_{10} = ()_2$

ii. $(110110.011)_2 = ()_{16}$

iii. $(231.36)_{10} = ()_2$

iv. $(11011.10)_2 = ()_{10}$

v. $(6FB.67)_{16} = ()_{10}$

vi. $(437)_8 = ()_{10}$

vii. $(1110111.11011)_2 = ()_8$

viii. $(1011.10110)_2 = ()_1$

ix. $(743.15)_8 = ()_2$

x. $(9AC.1B)_{16} = ()_2$

Solution:

i.

$$(73.12)_{10} = (?)_2$$

2		73	Remainder
2		36	1
2		18	0
2		9	0
2		4	1
2		2	0
2		1	0

msb ↑ LSB

$$(73)_{10} = (1001001)_2$$

For fractional part $\frac{12}{10}$ multiply the fractional part by new base and record the carry

$\cdot 13 \times 2 =$	0.26	$(\cdot 13)_{10} = (\cdot 0010)_2$
$\cdot 26 \times 2 =$	0.52	
$\cdot 52 \times 2 =$	1.04	
$\cdot 04 \times 2 =$	0.08	

(LSB)

So final answer

$$(73.12)_{10} = (1001001.0010)_2$$

ii.

$$\begin{array}{r} 00110115 \rightarrow 0110 \\ 0421 \ 0421 \cdot 0421 \\ = (36.6)_{16} \end{array}$$

iii.

$$(231.36)_{10}$$

Remainders

2	231	
2	115	1
2	57	1
2	28	1
2	14	0
2	7	0
2	3	1
	1	1

$$\begin{aligned} \cdot 36 \times 2 &= 0.72 \\ \cdot 72 \times 2 &= 1.44 \\ \cdot 44 \times 2 &= 0.88 \\ \cdot 88 \times 2 &= 1.76 \end{aligned}$$

$$(.36)_{10} = (.0101\ldots)_2$$

$$(231)_{10} = (11100111)_2$$

$$\text{Final answer } (231.36)_{10} = (11100111.0101\ldots)_2$$

$$(11011.10)_2$$

$$= 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2}$$

$$= (27.5)_{10}$$

$$(534)_{10}$$

$$= 5 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$$

$$= (340)_{10}$$

$$= 1536 + 240 + 11 + 375 + 0.273$$

$$= (1707.4023)_{10}$$

iv.

$$\begin{aligned}
 (437)_8 &= (?)_{10} \\
 &= 4 \times 8^2 + 3 \times 8^1 + 7 \times 8^0 \\
 &= 256 + 24 + 7 \\
 &= (287)_{10}
 \end{aligned}$$

v.

$$\begin{aligned}
 (1110111 \cdot 11011)_2 &= (?)_8 \\
 \text{Extra zero to make group of 3} \leftarrow \rightarrow & \\
 001110111 \cdot 110110 & \leftarrow \text{Extra zero to make group of 3} \\
 \underbrace{001}_{421} \underbrace{110}_{421} \underbrace{111}_{421} \cdot \underbrace{110}_{421} \underbrace{110}_{421} & \\
 &= (167.66)_8
 \end{aligned}$$

vi

$$\begin{aligned}
 (1011 \cdot 10110)_2 &= (?)_{16} \\
 \underbrace{1011}_{8421} \cdot \underbrace{1011}_{8421} \underbrace{0000}_{8421} & \\
 &= (B \cdot B0)_{16}
 \end{aligned}$$

vii.

$$(1011 \cdot 10110)_2 = (?)_{16}$$

$$\begin{array}{ccc} \overbrace{1011}^{\leftarrow} \cdot \overbrace{1011}^{\rightarrow} 0000 & & \\ \underbrace{0421} & \underbrace{0421} & \underbrace{0421} \\ = (B \cdot B0)_{16} \end{array}$$

viii.

$$(743 \cdot 15)_{10} = (?)_2$$

$$\begin{array}{ccccccc} & 7 & 4 & 3 & \cdot & 1 & 5 \\ & \swarrow & \downarrow & \searrow & & \swarrow & \searrow \\ 421 & 421 & 421 & 421 & & 421 & 101 \\ 111 & 100 & 011 & 001 & & & \end{array}$$

$$= (1111000110001101)_2$$

ix.

$$(9AC \cdot 1B)_{16} = (?)_2$$

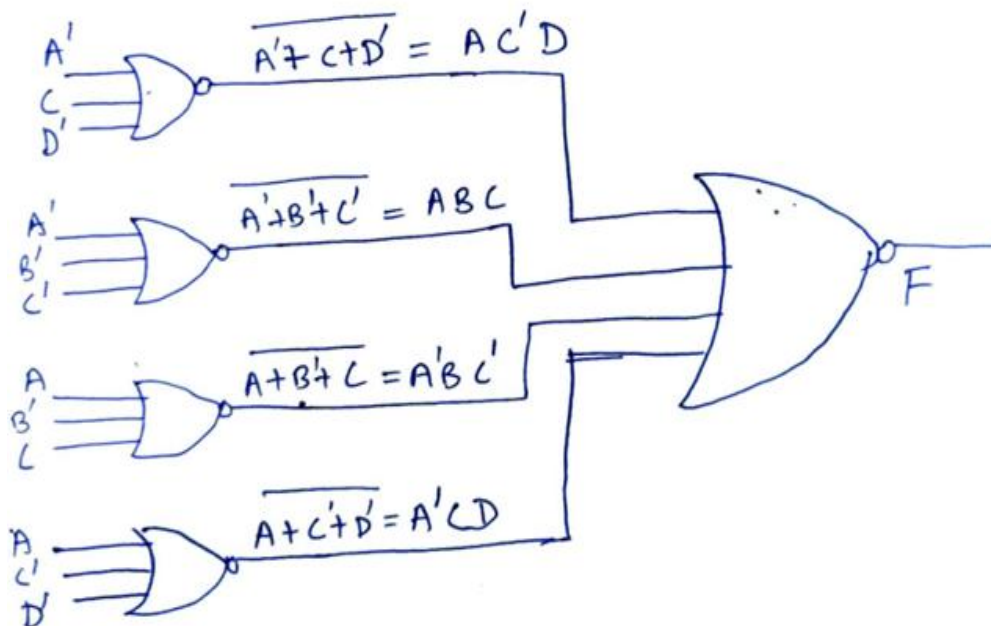
$$\begin{array}{ccccccc} (9) & (A) & (C) & \cdot & (1) & (B) & 16 \\ \swarrow & \swarrow & \downarrow & & \swarrow & \searrow & \\ 0421 & 0421 & 0421 & & 0421 & 0421 & \\ 1001 & 1010 & 1100 & & 0001 & 1011 & \\ = (100110101100 \cdot 00011011)_2 \end{array}$$

2. Using K-map method, determine the minimal POS expression for the following using decimal notation $F(A, B, C, D) = \pi M(3, 4, 5, 7, 9, 13, 14, 15)$. d(0, 2, 8). Also, implement using NOR gates only. [AKTU, odd sem, 2021-22]

Solution:

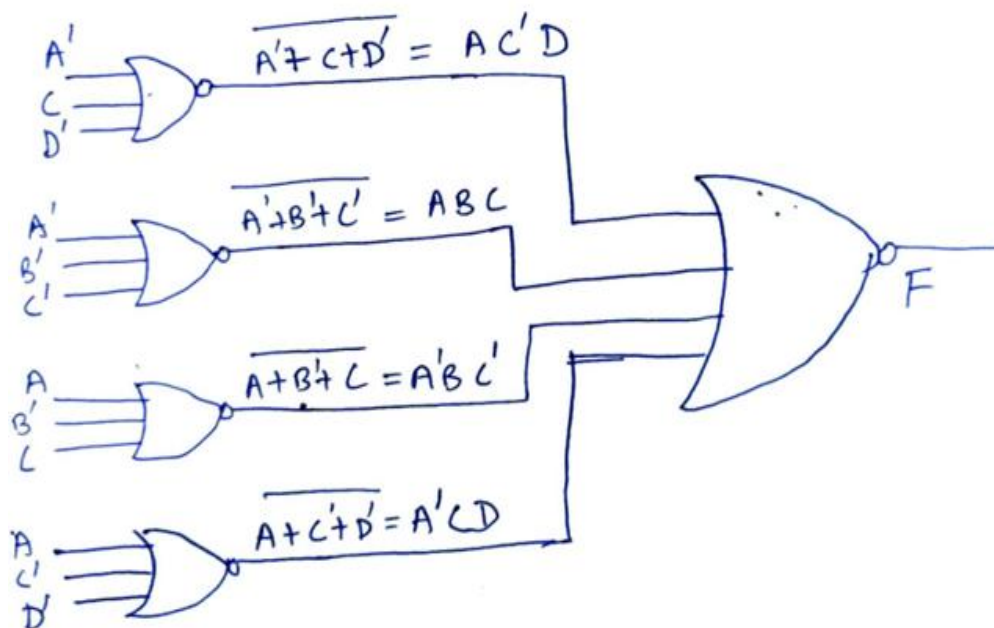
AB \ CD	C+D	C+D̄	C̄+D̄	C̄+D
A+B	X ⁰		0 ³	X ²
A+B̄	0 ⁴	0 ⁵	0 ⁷	
Ā+B̄		0 ¹³	0 ¹⁵	0 ¹⁴
Ā+B	X ⁸	0 ⁹		

$$f = (A + \bar{C} + \bar{D})(A + \bar{B} + C)(\bar{A} + \bar{B} + \bar{C})(\bar{A} + C + \bar{D})$$



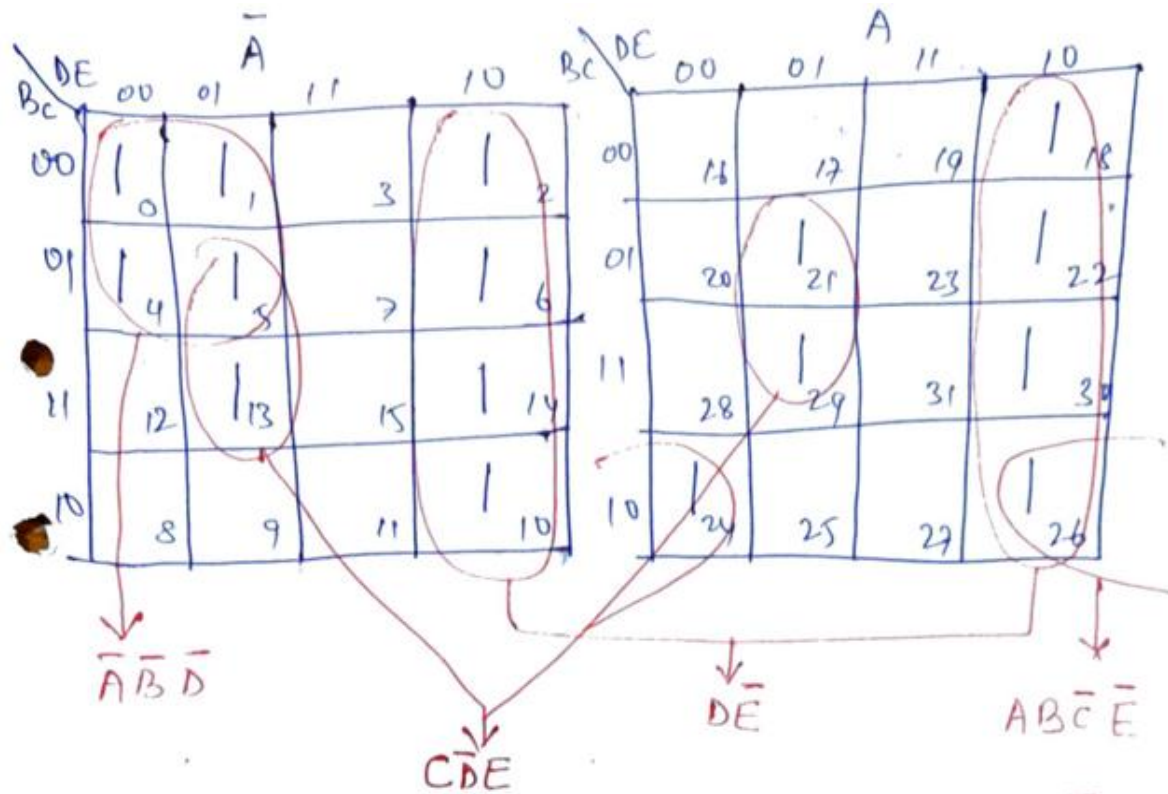
AB \ CD	C+D	C+D̄	C̄+D	C̄+D̄
A+B	X ⁰		0 ³	X ²
A+B̄	0 ⁴	0 ⁵	0 ⁷	
Ā+B		0 ¹³	0 ¹⁵	0 ¹⁴
Ā+B̄	X ⁸	0 ⁹		

$$f = (A + \bar{C} + \bar{D})(A + \bar{B} + C)(\bar{A} + \bar{B} + \bar{C})(\bar{A} + C + \bar{D})$$

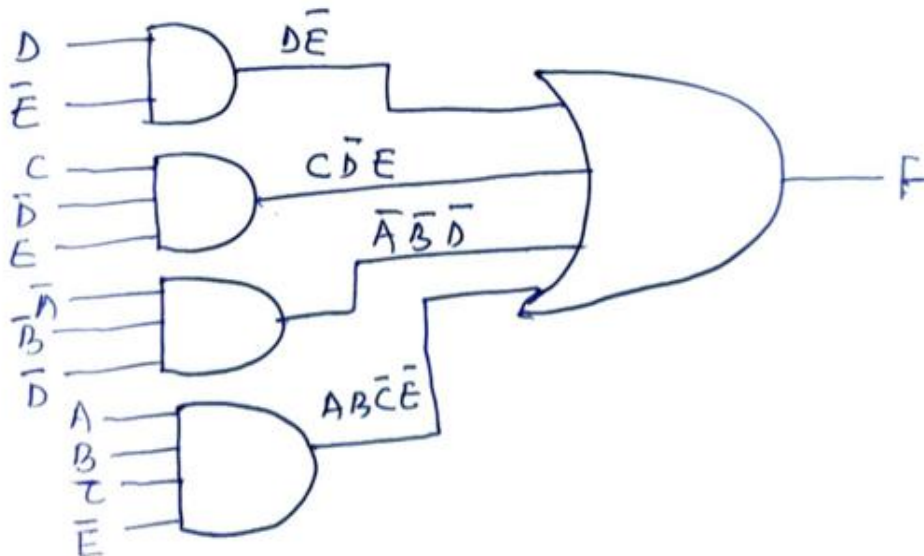


3. Simplify the expression $F(a,b,c,d,e) = \sum m(0,1,2,4,5,6,10,13,14,18,21,22,24,26,29,30)$ using K-map. Also, implement using logic gates. [AKTU, odd sem, 2021-22]

Solution:



$$F = D\bar{E} + C\bar{D}E + \bar{A}\bar{B}\bar{D} + A\bar{B}\bar{C}\bar{E}$$



4. Simplify the following function using K map:

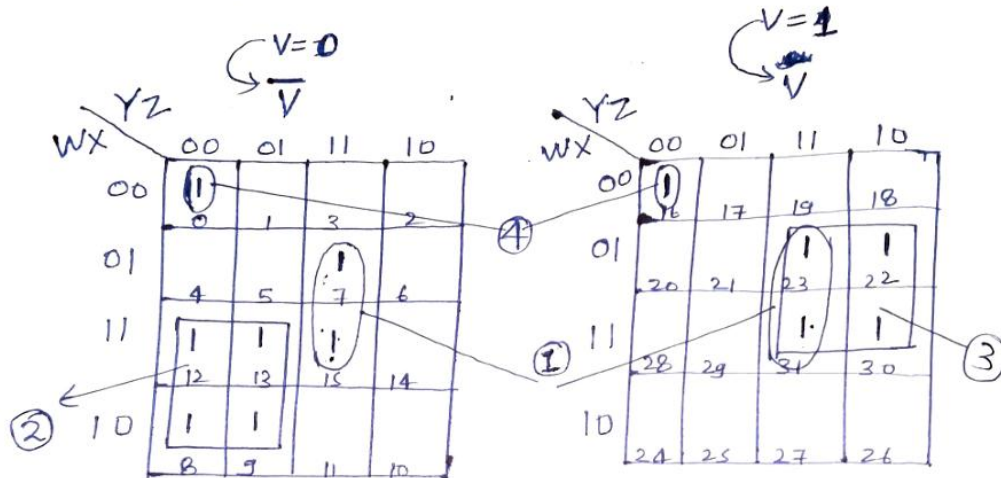
$$f(V, W, X, Y, Z) = \sum m(0, 7, 8, 9, 12, 13, 15, 16, 22, 23, 30, 31). \text{ [AKTU, even sem, 2022-23]}$$

Solution:

Solution

Total Variables = 5

$$\therefore \text{No. of boxes} = 2^5 = 32$$



Now for (1) $\rightarrow XYZ$

(2) $\rightarrow \bar{V}W\bar{Y}$

(3) $\rightarrow VX\bar{Y}$

(4) $\rightarrow \bar{W}\bar{X}\bar{Y}\bar{Z}$

Now Final Expression

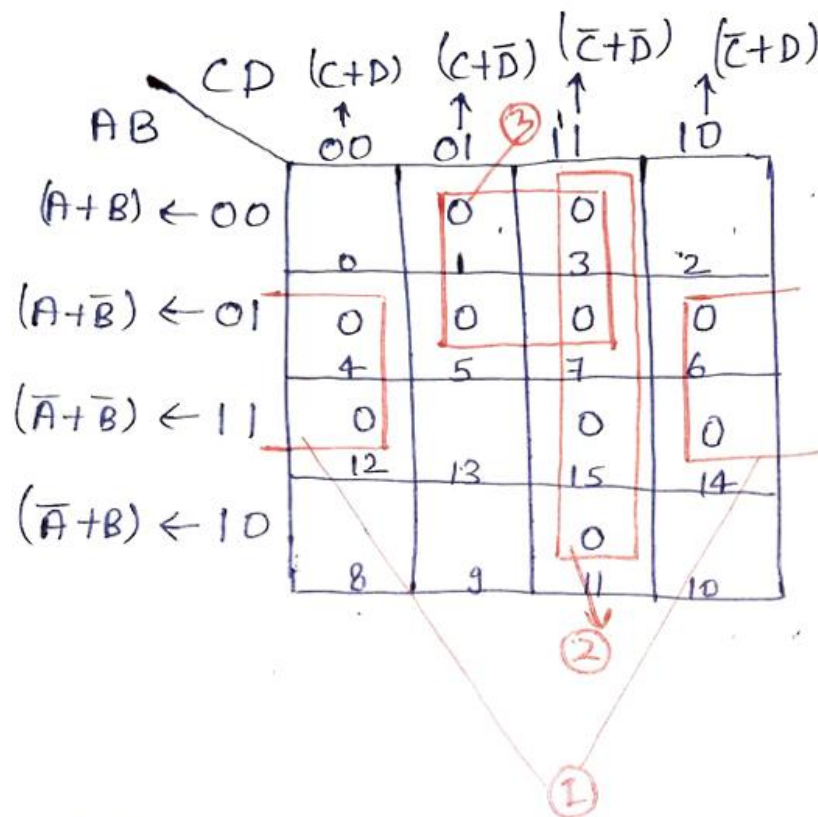
$$f(V, W, X, Y, Z) = (1) + (2) + (3) + (4)$$

$$f(V, W, X, Y, Z) = XYZ + \bar{V}W\bar{Y} + VX\bar{Y} + \bar{W}\bar{X}\bar{Y}\bar{Z}$$

5. Simplify the following function using K map and implement the simplified function using NOR gate only $F(A, B, C, D) = \pi M(1, 3, 4, 5, 6, 7, 11, 12, 14, 15)$. [AKTU, even sem, 2022-23]

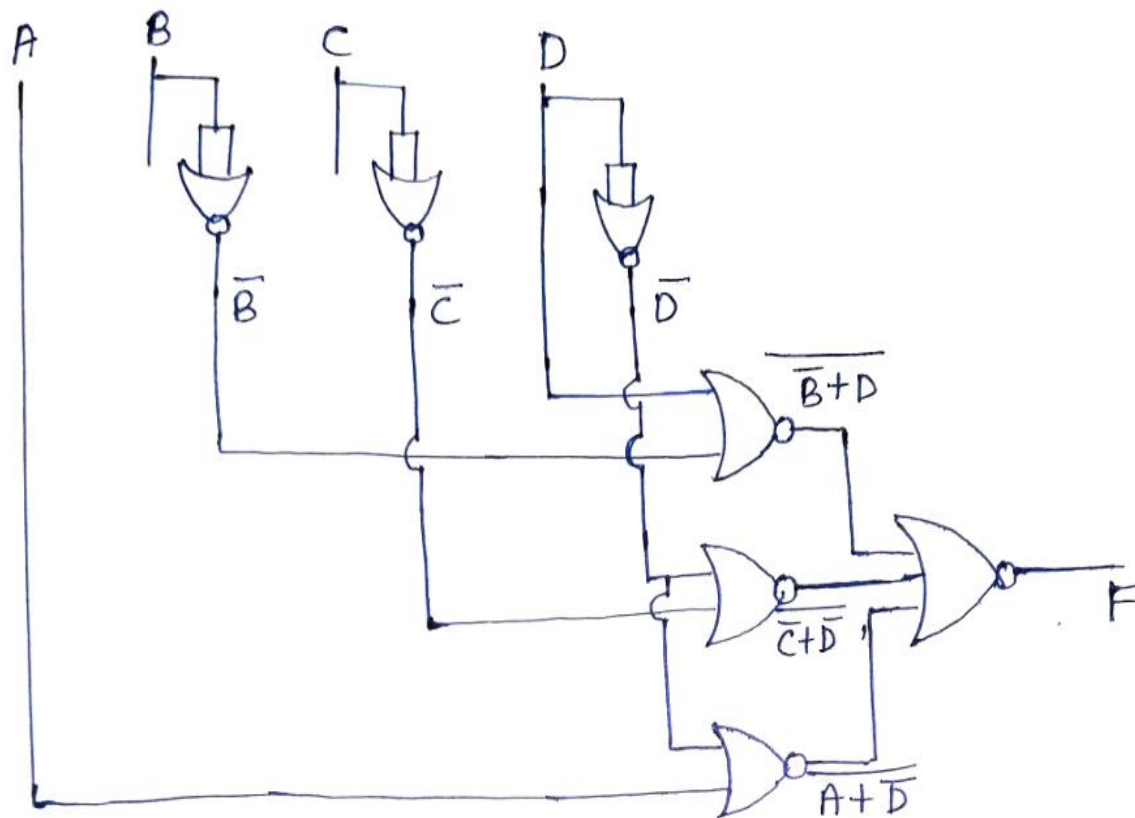
Solution:

soln
Total Variables = 4; $\therefore$ No. of boxes = $2^4 = 16$



$$\therefore F(A, B, C, D) = (\bar{B} + D) \cdot (\bar{C} + \bar{D}) \cdot (A + \bar{D})$$

Implementation using NOR GATE



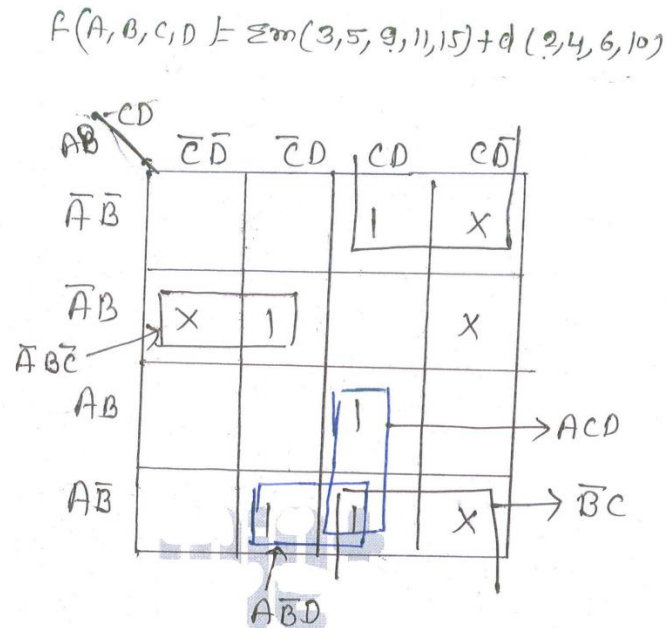
$$F = \overline{(\overline{B+D}) \cdot (\overline{C+D}) \cdot (\overline{A+D})}$$

$$F = \overline{(\overline{B+D}) \cdot (\overline{C+D}) \cdot (\overline{A+D})}$$

$$\boxed{F = (\overline{B+D}) \cdot (\overline{C+D}) \cdot (\overline{A+D})} \left\{ \text{using De-Morgan} \right\}$$

6. Simplify the following using K-map. $F(A, B, C, D) = \sum m(3, 5, 9, 11, 15) + d(2, 4, 6, 10)$. [AKTU, odd sem, 2011-12]

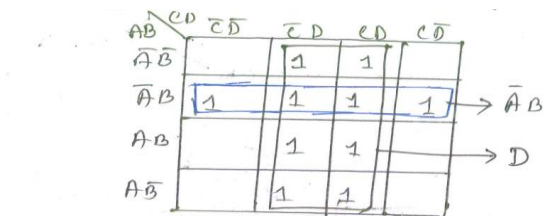
Solution:



So $F = \bar{A}\bar{B}D + A\bar{B}D + \bar{B}C + ACD$

7. Reduce the following function using K-Map. Also, implement with basic gates only.
 $F(A, B, C, D, E) = \sum m(1, 3, 4, 5, 6, 7, 9, 11, 13, 15)$. [AKTU, odd sem, 2020-21]

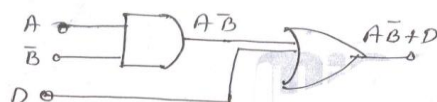
Solution:



$Y = \bar{A}\bar{B} + D$

Basic gates realization:-

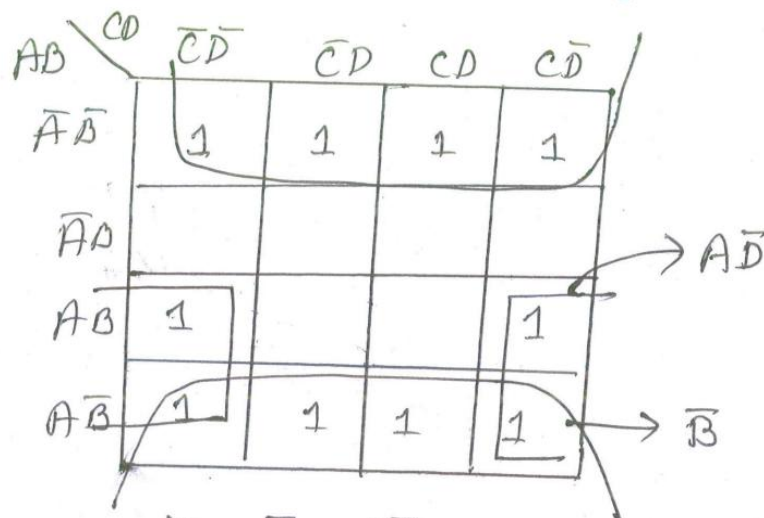
$Y = \bar{A}\bar{B} + D$



8. Simplify following logic expression using K-Map and realize using NOR gates only.
 $F(A, B, C, D, E) = \bar{A}\bar{B}\bar{C} + A\bar{C}\bar{D} + A\bar{B} + ABC\bar{D} + \bar{A}\bar{B}C$ [AKTU, even sem, 2013-14]

Solution:

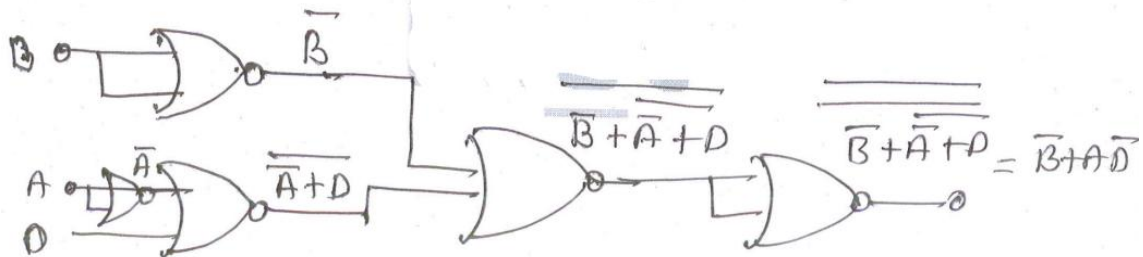
$$\begin{aligned}
 F(A, B, C, D) &= \bar{A}\bar{B}\bar{C} + A\bar{C}\bar{D} + A\bar{B} + ABC\bar{D} + \bar{A}\bar{B}C \\
 F(A, B, C, D) &= \bar{A}\bar{B}\bar{C}(D+\bar{D}) + A\bar{C}\bar{D}(B+\bar{B}) + A\bar{B}(C+\bar{C})(D+\bar{D}) + ABC\bar{D} \\
 &\quad + \bar{A}\bar{B}C(D+\bar{D}) \\
 &= \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D} + A\bar{C}\bar{D}B + \underline{A\bar{B}\bar{C}\bar{D}} + A\bar{B}CD + A\bar{B}C\bar{D} \\
 &\quad + \underline{A\bar{B}\bar{C}D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D} + \bar{A}\bar{B}CD + A\bar{B}C\bar{D}
 \end{aligned}$$



$$Y = \bar{B} + \bar{A}\bar{D}$$

NOR realization

$$\begin{aligned}
 Y &= \bar{B} + \bar{A}\bar{D} \\
 &= \overline{\overline{\bar{B} + \bar{A}\bar{D}}} = \overline{\bar{B} + \bar{A}\bar{D}} = \bar{B} + \bar{\bar{A}} + \bar{\bar{D}} = \bar{B} + A + D
 \end{aligned}$$



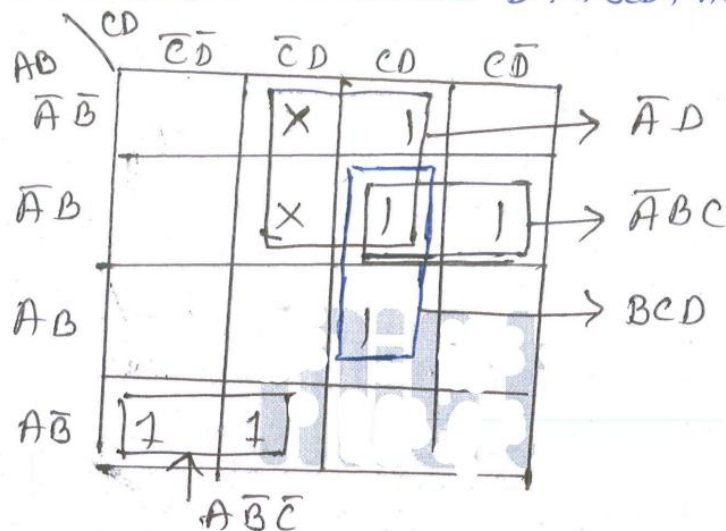
9. Minimize the following using K-map:

$$F(A, B, C, D) = AB'C' + A'BC + A'B'CD + ABCD + d(1, 5). \text{ [AKTU, odd sem, 2011-12]}$$

$$F(A, B, C, D) = A\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}\bar{B}CD + AB\bar{C}D + d(1, 5)$$

$$F(A, B, C, D) = A\bar{B}\bar{C}(D + \bar{D}) + \bar{A}B\bar{C}(D + \bar{D}) + \bar{A}\bar{B}CD + AB\bar{C}D$$

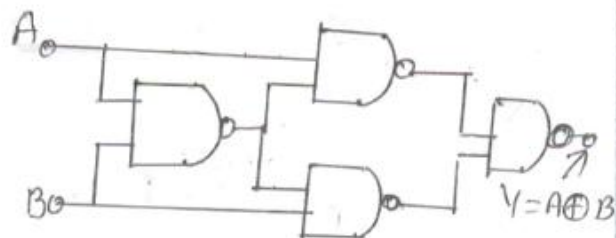
$$= A\bar{B}\bar{C}D + A\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}D + \bar{A}B\bar{C}\bar{D} + \bar{A}\bar{B}CD + AB\bar{C}D + d(1, 5)$$



$$F = A\bar{B}\bar{C} + BCD + \bar{A}BC + \bar{A}D$$

10. Implement EX-OR and EX-NOR gates using minimum NAND gates.

$$\begin{aligned} \text{O/p of Ex-or gate} &= \bar{A}B + A\bar{B} \\ &= A\bar{A} + A\bar{B} + \bar{A}B + B\bar{B} \\ &= A(\bar{A} + B) + B(\bar{A} + \bar{B}) \\ &= A(\bar{A} \cdot B) + B(\bar{A} \cdot \bar{B}) \\ &= \overline{A \cdot (\bar{A} \cdot B)} + \overline{B \cdot (\bar{A} \cdot \bar{B})} \\ &= \overline{A \cdot (\bar{A} \cdot B)} \cdot \overline{B \cdot (\bar{A} \cdot \bar{B})} \end{aligned}$$



- NAND as EX-NOR gate.

o/b of EX-NOR gate

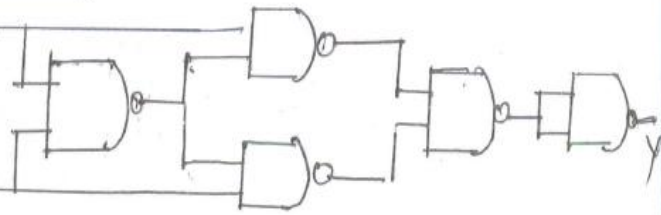
$$= AB + \bar{A}\bar{B}$$

It is opposite of AND

EX-OR gate. So

by adding one inverter gate we get the

EX-NOR gate.



11. Implement EX-OR and EX-NOR gates using minimum NOR gates.

o/b of EX-OR gate

$$Y = \bar{A}B + A\bar{B}$$

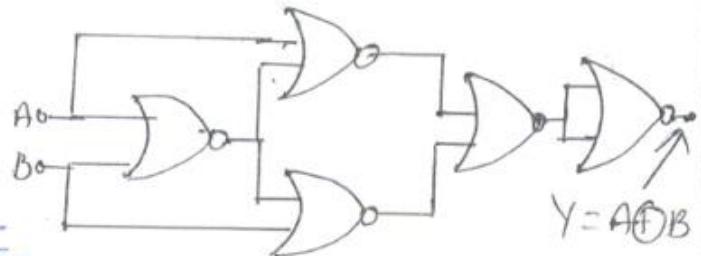
$$= \bar{A}B + \bar{A}\bar{B} + A\bar{A} + B\bar{B}$$

$$= \bar{A}(A+B) + \bar{B}(A+B)$$

$$= \bar{A}(A+B) + \bar{B}(A+B)$$

$$= \bar{A} + (\bar{A} + \bar{B})(A+B)$$

$$= \bar{A} + (\bar{A} + \bar{B}) + B + (A+B)$$



- NOR as EX-NOR gate :

o/b of EX-NOR gate

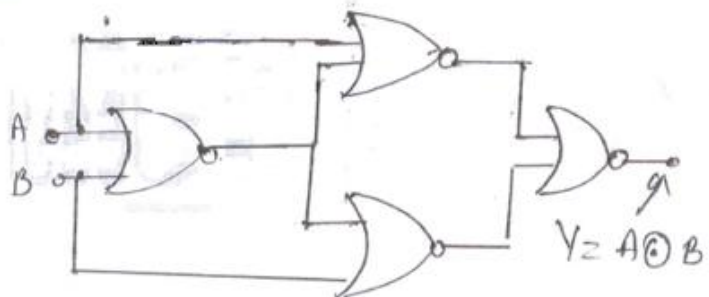
$$Y = AB + \bar{A}\bar{B}$$

It is opposite of

EX-OR gate. So,

by adding one inverter we get the

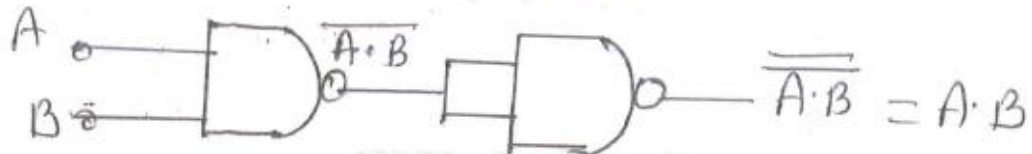
EX-NOR gate.



12. Define Universal Gates. Implement AND, OR, NOR by using NAND gates only.

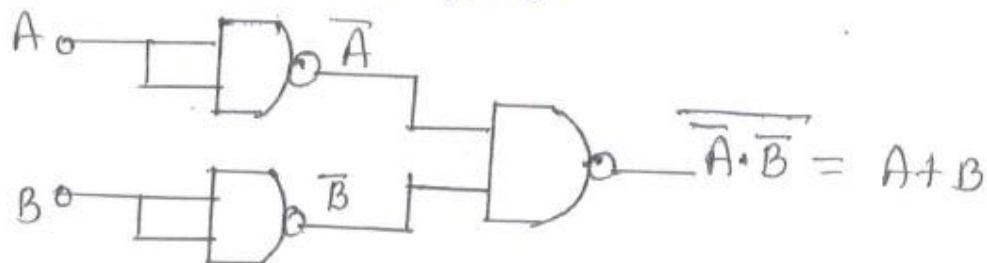
NAND as AND gate

$$\begin{aligned} \text{O/p of AND gate } Y &= A \cdot B \\ &= \overline{\overline{A \cdot B}} \end{aligned}$$



NAND as OR Gate

$$\begin{aligned} \text{O/p of OR gate} &= A + B \\ &= \overline{\overline{A + B}} \\ &= \overline{\overline{A} \cdot \overline{B}} \end{aligned}$$



NAND as NOR gate.

$$\begin{aligned} \text{O/p of NOR gate} &= \overline{A + B} \\ &= \overline{\overline{\overline{A + B}}} \\ &= \overline{\overline{A} \cdot \overline{B}} \end{aligned}$$

